

CS3313计算理论 课程笔记

uu-matter543

2025–2026–1

目录

1	自动机理论	3
1.1	有限状态机与正则语言	3
1.1.1	确定有限状态机	3
1.1.2	非确定有限状态机	3
1.1.3	正则完备的语言运算	5
1.1.4	正则语言的表达式	6
1.1.5	正则语言的可泵性	7
1.1.6	正则语言与可计算理论	8
1.2	上下文无关语言与下推自动机	9
1.2.1	上下文无关文法	9
1.2.2	下推自动机	10
1.2.3	上下文无关语言的可泵性	12
1.2.4	上下文无关完备的语言运算	13
1.2.5	上下文无关语言与可计算理论	14
1.2.6	(*)确定下推自动机	14
2	可计算理论	15
2.1	丘奇图灵论题	15
2.1.1	图灵机模型	15
2.1.2	变种图灵机	17
2.1.3	算法的理论基础	19
2.2	可判定性	20
2.2.1	正则语言与可判定性	20
2.2.2	上下文无关语言与可判定性	20
2.2.3	不可判定性	20

2.3	归约	22
2.3.1	其它不可判定的语言	22
2.3.2	涉及计算历史的归约	23
2.3.3	映射归约	24
3	复杂性理论	27
3.1	时间复杂度	27
3.1.1	时间复杂度的表述	27
3.1.2	P 类	27
3.1.3	NP 类	28
3.1.4	NP 完全性	28
3.2	空间复杂性	34
3.2.1	空间复杂度的表述	34
3.2.2	超线性空间复杂度类	34
3.2.3	PSPACE 完全性	35
3.2.4	亚线性空间复杂度类	37
3.2.5	NL 完全性	38
4	(*)计算理论中的其他论题	41
4.1	分层定理	41
4.1.1	空间分层定理	41
4.1.2	时间分层定理	41
4.2	对角线方法的局限性	42
4.3	逻辑电路复杂性	42

1 自动机理论

1.1 有限状态机与正则语言

1.1.1 确定有限状态机

确定有限状态机 (DFA = Deterministic Finite Automata) 是 5 元组 $(Q, \Sigma, \delta, q_0, F)$

- Q 为机器状态的有限集
- Σ 为可能输入字符的有限集
- $\delta : Q \times \Sigma \rightarrow Q$ 为状态转移函数
- $q_0 \in Q$ 为机器的初状态
- $F \subseteq Q$ 为接受状态集

接受: 设 M 为 DFA, $w = w_1w_2w_3 \dots w_n$ 为输入字符串, 其中 $w_i \in \Sigma$, 若向 M 输入 w 后机器的最终状态 $q \in F$, 则称 M 接受 (accepts) w , 即:

$$M \text{ accepts } w \Leftrightarrow \begin{cases} r_0 = q_0 \\ \delta(r_{n-1}, w_i) = r_i \\ r_n \in F \end{cases}$$

识别: 若 $A = \{w | M \text{ accepts } w\}$, 称 M 识别 (recognizes) A , 记作 $A = L(M)$

对给定的 DFA M , $L(M)$ 必然唯一, 但不一定有限

正则: 若 $\exists$ DFA $M, S = L(M)$, 则称语言 S 是正则的, 或称 S 是正则语言

对给定的正则语言 S , M 不唯一

1.1.2 非确定有限状态机

非确定有限状态机 (NFA = Non-eterministic Finite Automata) 是 5 元组 $(Q, \Sigma, \delta, q_0, F)$

- Q 为机器状态的有限集
- Σ 为可能输入字符的有限集
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$ ¹ 为状态转移函数, 其中 ε 表示空字符
- $q_0 \in Q$ 为机器的初状态
- $F \subseteq Q$ 为接受状态集

与 DFA 相比, NFA 在状态转移函数方面有所不同, 具体有:

- 处于某些状态时可接受空字符作为输入
- 某些状态下可能不接受某些字符作为输入
- 某些状态下输入某些字符有多个可能的状态

¹ $P(Q)$ 表示 Q 的幂集, 即 $P(Q) = \{X | X \subseteq Q\}$

接受: 设 M 为 NFA, $w = w_1w_2w_3 \dots w_n$ 为输入字符串, 其中 $w_i \in \Sigma$, 若向 M 输入 w , 其中可插入若干个 ε , 且若有多个可能的新状态则可任意选择, 使得每个字符输入到机器上都有效, 且机器的最终状态 $q \in F$, 则称 M 接受 (accepts) w

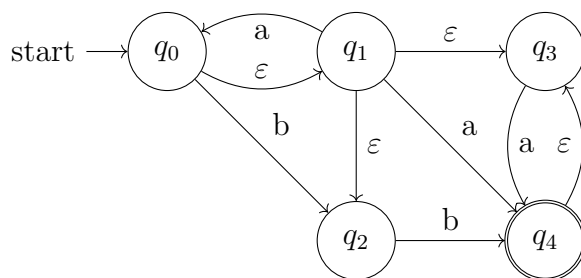
识别: 若 $A = \{w | M \text{ accepts } w\}$, 称 M 识别 (recognizes) A , 记作 $A = L(M)$

定理 $\forall M \in \text{NFA}, \exists N \in \text{DFA}, L(M) = L(N)$

由 NFA $N(Q, \Sigma, \delta, q_0, F)$ 构造 DFA 的过程如下:

- $\forall q \in Q$, 定义 q 的 ε 闭包 $E(q)$ 为: NFA 状态为 q 时接受任意数量 (可以为 0) 的 ε 可到达的状态
- 构造 DFA $D(P(Q), \Sigma, \delta', E(q_0), F')$ 使得 $L(D) = L(M)$
 - $\delta'(R, a) = \bigcup_{r \in R} \{\delta(r, a)\}$
 - $F' = \{R \in P(Q) | R \cap F \neq \emptyset\}$

例 将如下 NFA 转化为 DFA



解 列出每个状态的 ε 闭包有:

- $E(q_0) = \{q_0, q_1, q_2, q_3\}$
- $E(q_1) = \{q_1, q_2, q_3\}$
- $E(q_2) = \{q_2\}, E(q_3) = \{q_3\}$
- $E(q_4) = \{q_3, q_4\}$

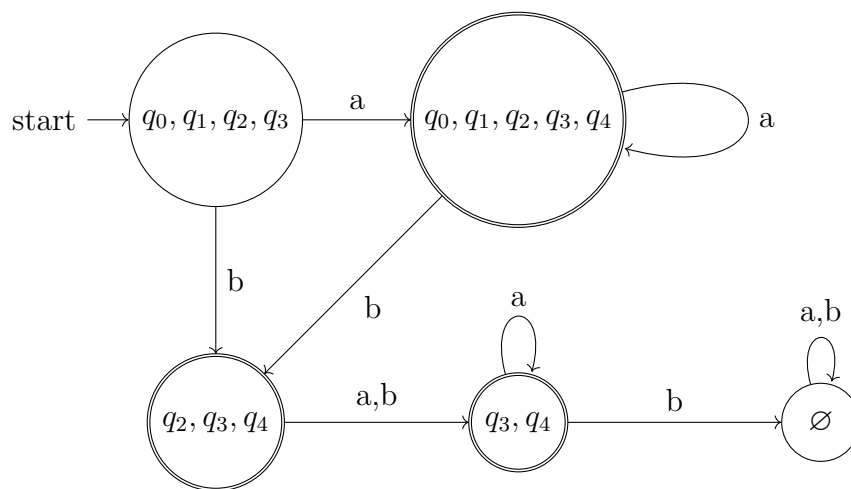
构造初态 $q'_0 = \{q_0, q_1, q_2, q_3\}$, 然后求解 $\delta'(q'_0, a)$ 和 $\delta'(q'_0, b)$, 并递归地对产生的新状态类似求解, 即可得到等价的 DFA

- $\delta'(q'_0, a): \delta(q_1, a) = \{q_0, q_4\}, E(q_0) \cup E(q_4) = Q$, 构造新状态 $q'_1 = Q = \delta'(q'_0, a)$
- $\delta'(q'_0, b): \delta(q_0, b) = \{q_2\}, \delta(q_2, b) = \{q_4\}, E(q_2) \cup E(q_4) = \{q_2, q_3, q_4\}$, 构造新状态 $q'_2 = \{q_2, q_3, q_4\} = \delta'(q'_0, b)$
- $\delta'(q'_1, a): \delta(q_1, a) = \{q_0, q_4\}, E(q_0) \cup E(q_4) = Q$, 有 $\delta'(q'_1, a) = q'_1$
- $\delta'(q'_1, b): \delta(q_0, b) = \{q_2\}, \delta(q_2, b) = \{q_4\}, E(q_2) \cup E(q_4) = \{q_2, q_3, q_4\}$, 有 $\delta'(q'_1, b) = q'_2$
- $\delta'(q'_2, a): \delta(q_3, a) = \{q_4\}, E(q_4) = \{q_3, q_4\}$, 构造新状态 $q'_3 = \{q_3, q_4\} = \delta'(q'_2, a)$
- $\delta'(q'_2, b): \delta(q_3, b) = \{q_4\}, E(q_4) = \{q_3, q_4\}$, 所以 $\delta'(q'_2, b) = q'_3$
- $\delta'(q'_3, a): \delta(q_3, a) = \{q_4\}, E(q_4) = \{q_3, q_4\}$, 所以 $\delta'(q'_3, a) = q'_3$

• $\delta'(q'_3, b) = \emptyset$: 构造新状态 $q'_4 = \emptyset = \delta'(q'_3, b)$

• $\delta'(q'_4, a) = \delta'(q'_4, b) = \emptyset = q'_4$

构造得到的 DFA 如下:



1.1.3 正则完备的语言运算

语言的运算: 设 A 和 B 是语言, 则:

- 并集 (union): $A \cup B = \{w | w \in A \vee w \in B\}$
- 复合/连接 (concatenation): $A \circ B = \{xy | x \in A \wedge y \in B\}$
- 克莱尼星 (Kleene star): $A^* = \{x_1 x_2 x_3 \cdots x_k | k \geq 0 \wedge (\forall i, x_i \in A)\}$
- 交集 (intersection): $A \cap B = \{w | w \in A \wedge w \in B\}$
- 补集 (complement): $\bar{A} = \{w | w \in \Sigma^* \wedge w \notin A\}$

注意: A^* 包含空串 ε

定理 若 A, B 都是正则语言, 则 $A \cup B, A \circ B, A^*, A \cap B, \bar{A}$ 都是正则语言

正则完备性证明

设 DFA $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 识别 A , DFA $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ 识别 B

证明: $A \cup B$ 是正则语言

构造 NFA $(\{q_0\} \cup Q_1 \cup Q_2, \Sigma, \delta, q_0, F_1 \cup F_2)$ 识别 $A \cup B$, 其中

$$\delta(q, a) = \begin{cases} \{q_1, q_2\} & q = q_0 \wedge a = \varepsilon \\ \emptyset & q = q_0 \wedge a \neq \varepsilon \\ \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

证明： $A \circ B$ 是正则语言

构造 $\text{NFA}(Q_1 \cup Q_2, \Sigma, \delta, q_1, F_2)$ 识别 $A \circ B$, 其中

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in (Q_1 - F_1) \vee (q \in F_1 \wedge a \neq \varepsilon) \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \wedge a = \varepsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

证明： A^* 是正则语言

构造 $\text{NFA}(Q_1 \cup \{q_0\}, \Sigma, \delta, q_0, F_1 \cup \{q_0\})$ 识别 A^* , 其中

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in (Q_1 - F_1) \vee (q \in F_1 \wedge a \neq \varepsilon) \\ \delta_1(q, a) \cup \{q_0\} & q \in F_1 \wedge a = \varepsilon \\ \{q_1\} & q = q_0 \wedge a = \varepsilon \\ \emptyset & q = q_0 \wedge a \neq \varepsilon \end{cases}$$

证明： $\bar{A}$ 是正则语言

构造 $\text{DFA } M = (Q_1, \Sigma, \delta_1, q_1, Q_1 - F_1)$ 识别 $\bar{A}$, 即证

证明： $A \cap B$ 是正则语言

证法 1: 构造 $\text{DFA } M = (Q_1 \times Q_2, \Sigma, \delta, \langle q_1, q_2 \rangle, F_1 \times F_2)$ 识别 $A_1 \cap A_2$, 其中 $\delta(\langle r_1, r_2 \rangle, a) = \langle \delta_1(q_1, a), \delta_2(q_2, a) \rangle$, 即证

证法 2: 利用已经证明过的正则完备运算, 有 A, B 是正则语言 $\Rightarrow \bar{A}, \bar{B}$ 是正则语言 $\Rightarrow \overline{\bar{A} \cup \bar{B}}$ 是正则语言 $\Rightarrow \overline{\bar{A} \cup \bar{B}} = A \cap B$ 是正则语言

例 设 $A/B = \{w | wx \in A \wedge x \in B\}$, 证明: 若 A 是正则语言, 则 A/B 是正则语言

解 设 $\text{DFA } M = (Q, \Sigma, \delta, q, F)$ 可识别 A

对 $\forall q_k \in Q$, 若 $\exists s \in B$, 机器的状态为 q_k 时输入 s 使得机器的状态变为 F , 则令 $q_k \in F'$, 最终 $(Q, \Sigma, \delta, q, F')$ 即为识别 A/B 的 DFA

1.1.4 正则语言的表达式

正则表达式： 给定 Σ , 语言 R 是正则语言当且仅当 R 满足以下条件之一：

- $R = \{a\}$, 其中 $a \in \Sigma$
- $R = \{\varepsilon\}$
- $R = \emptyset$
- $R = R_1 \cup R_2$, 其中 R_1, R_2 是正则语言

- $R = R_1 \circ R_2$, 其中 R_1, R_2 是正则语言
- $R = R_1^*$, 其中 R_1 是正则语言

证明：正则表达式可由 NFA 识别

前 3 个条件可构造识别它们的 NFA 证明：



图 1: 识别 $\{a\}, \{\varepsilon\}, \emptyset$ 的 NFA

其余条件则可根据正则语言的运算封闭性证明

证明：DFA 识别的语言能用正则表达式表示

直观而言，可用 $\cup, \circ, ^*$ 合并状态直到只剩 2 个状态

严格证明：用 $1, 2, 3, \dots, n$ 标记 DFA M 的状态，定义字符串 $R(i, j, k)$ 为：机器状态为 q_i 时，输入 $R(i, j, k)$ 可到达 q_j ，且路径 $path(i, j)$ 满足 $\forall q_x \in path, x \leq k$

设初状态为 q_1 ，则 $L(M) = \{R(1, j, n) | q_j \in F\}$ ，且有

$$R(i, j, k) = R(i, j, k-1) \cup [R(i, k, k-1) \circ R(k, k, k-1)^* \circ R(k, j, k-1)]$$

可对 $R(i, j, k)$ 进行归纳，即证

1.1.5 正则语言的可泵性

定理 正则语言都是可泵的，即：

设 DFA $M = (Q, \Sigma, \delta, q, F)$, $A = L(M)$, $s \in A$ ，则 $\exists p, \forall |s| > p$, s 可表示为 $s = xyz$ 且满足： $(|y| > 0) \wedge (|xy| \leq p) \wedge (\forall i \in \mathbb{N}, xy^i z \in A)$

注意：如上可泵性是正则语言的必要条件而非充分条件

正则语言可泵性的证明

设 $|xyz| > |Q|$ ，则对于 DFA 运行的状态序列 $q_0 q_1 q_2 \dots q_n$ ，有 $\exists i \neq j, q_i = q_j$
w.l.o.g. 设 $i < j$ ，在该运行过程中重复 q_i 到 q_{j-1} 的字符串不影响结果，即证

例 证明： $A = \{w \in \{0, 1\}^* | w \text{ has equal 0s and 1s}\}$ 不是正则语言

解 $0^p 1^p \in A \because |0^p 1^p| = 2p > p \therefore$ 泵引理中 xy 是 0^p 的子串

设 $y = 0^n$ ，则 $xy^0 z = 0^{p-n} 1^p \notin A \therefore A$ 不是正则语言

1.1.6 正则语言与可计算理论

给定有限状态机模型，可解决如下 3 类问题：

- 可接受问题：某个字符串是否属于某个正则语言，只需将该字符串输入到有限状态机中
- 判空问题：某个语言是否为空，可利用图遍历算法，将状态转移函数构造为图，并搜索初状态到接受状态的路径
- 等价问题：给定的多个正则语言是否等价，可根据 $A = B \Leftrightarrow (\overline{A} \cap B) \cup (A \cap \overline{B}) = \emptyset$ 构造识别语言 $(\overline{A} \cap B) \cup (A \cap \overline{B})$ 的有限状态机，并运行判空算法即可

1.2 上下文无关语言与下推自动机

1.2.1 上下文无关文法

上下文无关文法 (CFG = Context Free Grammar) 是 4 元组 (V, Σ, R, S)

- V 是非终结符集, 是需要重写的字符集合
- Σ 是终结符集, 是不能再重写的字符集合
- $R: V \rightarrow (V \cup \Sigma)^*$ 是重写规则
- $S \in V$ 是初始字符

上下文无关语言 (CFL = Context Free Language) 是指根据给定的 CFG, 由初始字符开始, 经过有限次重写, 能够得到的所有只含终结符的字符串的集合

“上下文无关”的由来: 只要某个字符在中途出现就可以直接重写, 无需考虑相邻字符

例 写出 $A = \{0^n 1^n | n \geq 0\}, B = \{1^n 0^n | n \geq 0\}, C = A \cup B$ 的 CFG

解 $A: S \rightarrow 0S1 | \varepsilon \quad B: S \rightarrow 1S0 | \varepsilon$
 $C: S \rightarrow S_1 | S_2 \quad S_1 \rightarrow 0S_1 1 | \varepsilon \quad S_2 \rightarrow 1S_2 0 | \varepsilon$

例 写出 $A = \{a^i b^j c^k | i = j \vee j = k\}$ 的 CFG

解 $S \rightarrow S' | S'' \quad S' \rightarrow S'_1 S'_2 \quad S'_1 \rightarrow aS'_1 b | \varepsilon \quad S'_2 \rightarrow cS'_2 | \varepsilon$
 $S'' \rightarrow S''_1 S''_2 \quad S''_1 \rightarrow aS''_1 | \varepsilon \quad S''_2 \rightarrow bS''_2 c | \varepsilon$

歧义性指: 若根据 CFG 生成相同目标字符串存在多种不同过程, 就称该 CFG 具有歧义性, 例如 $S \rightarrow S_1 | a, S_1 \rightarrow a$ 生成 a 的过程就有如下 2 种: $S \rightarrow a$ 和 $S \rightarrow S_1 \rightarrow a$

最左优先派生: 当某个中间串含有多个非终结符时, 必须先重写最左侧的非终结符

引入最左优先派生可部分消除歧义性, 但不能完全根除歧义性, 例如: $a \rightarrow a + a | a \times a | n$ 生成 $n + n \times n$ 的最左优先派生过程就有至少如下 2 种:

- $a \rightarrow a + a \rightarrow n + a \rightarrow n + a \times a \rightarrow n + n \times a \rightarrow n + n \times n$
- $a \rightarrow a \times a \rightarrow a + a \times a \rightarrow n + a \times a \rightarrow n + n \times a \rightarrow n + n \times n$

歧义性带来的问题: 无法根据 CFG 做出准确的解释, 违反了编程语言的无二义性

研究表明, 不存在可将所有歧义文法转化为非歧义文法的算法, 也不存在判定上下文无关文法是否为歧义文法的算法; 但存在某些技术, 可使设计出的程序语言是非歧义的

乔姆斯基范式 (Chomsky Normal Form) 是所有重写规则均满足如下形式之一的 CFG (其中大写字母表示非终结符, S 是初始字符, 小写字母表示终结符):

- $A \rightarrow BC (B \neq S, C \neq S)$, 允许 $B = A$ 或 $C = A$ (非终结符可重写为 2 个非终结符)
- $A \rightarrow a$ (非终结符可重写为单个终结符)
- $S \rightarrow \varepsilon$ (只有初始字符才可以重写为空字符)

定理 任意上下文无关文法均可重构为等价的乔姆斯基范式

乔姆斯基范式通用性证明

对于给定的 $CFG = (V, \Sigma, R, S)$ ，进行如下操作：

1. 添加重写规则 $S_0 \rightarrow S$ ，其中 S_0 是新 CFG 的初始字符
 2. 对于重写规则 $A \rightarrow \varepsilon$ ，将其它出现 A 的重写规则按照 A 取 ε 和保留 A 列举，从而将 $A \rightarrow \varepsilon$ 删除，例如：
 - $R \rightarrow uAv$ 改写为 $R \rightarrow uv|uAv$
 - $R \rightarrow uAvAw$ 改写为 $R \rightarrow uvw|uAvw|uvAw|uAvAw$
 - 若 $R \rightarrow \varepsilon$ 未被删除过，且存在重写规则 $R \rightarrow A$ ，则添加重写规则 $R \rightarrow \varepsilon$
 3. 若存在重写规则 $A \rightarrow B$ 且 B 不是终结符，则 $\forall B \rightarrow u$ ，添加 $A \rightarrow u$ ，删除 $A \rightarrow B$ ，之后引入额外变元消除 $A \rightarrow v_1v_2v_3 \cdots v_k$ 中 $k \geq 3$ 的情况，例如将其改写为 $A \rightarrow V_1A_1, V_1 \rightarrow v_1, A_1 \rightarrow V_2A_2, V_2 \rightarrow v_2, \dots, A_{k-2} \rightarrow V_{k-1}A_{k-1}, V_{k-1} \rightarrow v_{k-1}, A_{k-1} \rightarrow v_k$ ，使得所有从 A 出发的重写过程均符合乔姆斯基范式
 4. 重复执行第 2 步到第 4 步直到满足乔姆斯基范式
- 通过构造过程证明了乔姆斯基范式的通用性

定理 乔姆斯基范式生成长度为 $|w|$ 的字符串的过程需要 $2|w| - 1$ 步

该定理的证明详见 Homework 3-Problem 4

1.2.2 下推自动机

下推自动机 (PDA = Pushdown Automata) 是 6 元组 $(Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q 为机器状态的有限集
- Σ 为可能输入字符的有限集
- Γ 为栈中可能字符的有限集
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \rightarrow P(Q \times (\Gamma \cup \{\varepsilon\}))$ 为状态转移函数
- $q_0 \in Q$ 为机器的初状态
- $F \subseteq Q$ 为接受状态集

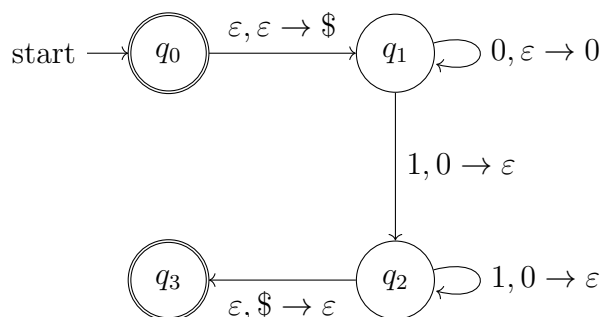
状态转移函数可描述为：根据当前状态、输入以及栈顶元素，改变状态和栈顶元素，其中输入和栈顶元素不必要参考，栈顶元素也不必要改变

接受： 设 M 为 PDA， $w = w_1w_2w_3 \dots w_n$ 为输入字符串，其中 $w_i \in \Sigma$ ，若向 M 输入 w ，其中可插入若干个 ε ，且若有多个可能的新状态则可任意选择，使得每个字符输入到机器上都有效，且机器的最终状态 $q \in F$ ，则称 M 接受 (accepts) w ，即 M 接受 F 当且仅当：对于字符串 $w = w_1w_2w_3 \cdots w_n$ ，存在状态序列 $q = q_0q_1q_2 \cdots q_n$ 及栈顶序列 $r = r_0r_1r_2 \cdots r_m$ ，其中 q_0 为初始状态， $r_0 = \varepsilon$ 为空栈顶，满足 $\forall i \in [1, m], (q_i, r_i) \in \delta(q_{i-1}, w_i, r_{i-1})$ 且 $q_m \in F$

识别： 若 $A = \{w | M \text{ accepts } w\}$ ，称 M 识别 (recognizes) A ，记作 $A = L(M)$

例 设计识别语言 $\{0^n 1^n | n \geq 0\}$ 的 PDA

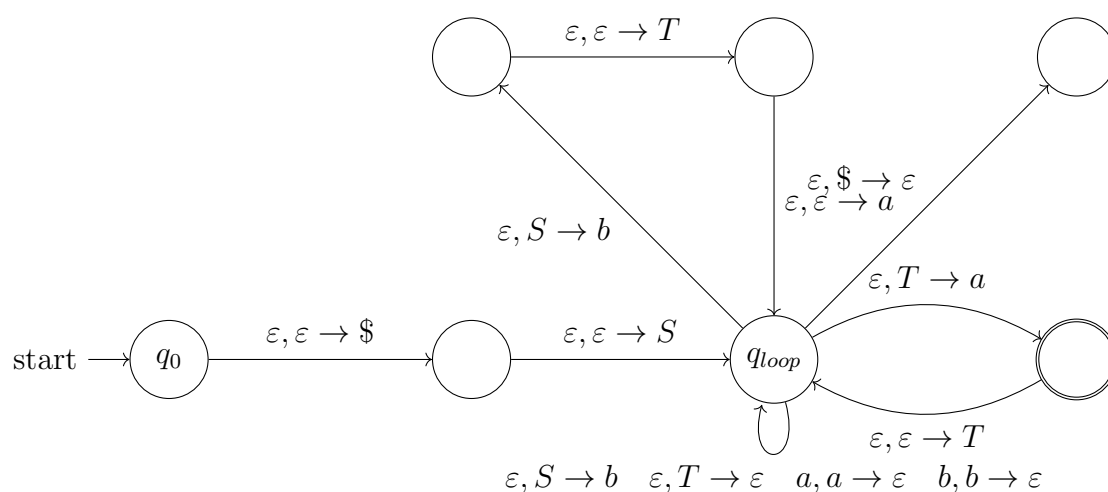
解 思路：将全部 0 入栈，之后遇到 1 时将 0 出栈；为了识别栈中没有 0 的情况，初始时将特殊字符 \$ 压入栈，并最终以此判断 0 是否全部出栈；设计出的 PDA 如下



定理 上下文无关文法构造出的语言与 PDA 能够识别的语言等价

根据 CFG 可构造 PDA 的证明

以实例 $S \rightarrow aTb | b$ $T \rightarrow Ta | \epsilon$ 说明，构造对应的 PDA，当栈顶为终结符时，就将读取输入进行比较；当栈顶为非终结符时，将其出栈，用任意重写规则替换，倒序入栈



引理 任意 PDA 可转化为满足如下所有条件的 PDA:

1. 存在唯一接受状态
2. 对于接受的输入，读取全部字符后，栈为空
3. 状态转移时，栈上的操作只能为入栈或出栈，不能单步改变栈顶字符或不进行栈操作

根据 PDA 可构造 CFG 的证明

对栈的高度和输入字符的状态转移次数进行递归：由于初始状态和最终接受状态栈均为空，则 PDA 接受 w 的过程只可能有 2 种情况：

- 全程栈非空，则设最先入栈（必然也是最后出栈）的字符为 b ，则生成形式为 $A \rightarrow Tb$ 的重写规则；之后将栈中只有 b 视为栈空，则栈的最大高度会降低 1
- 存在 n 个栈为空的中间状态，则生成形式为 $T \rightarrow A_0A_1A_2 \cdots A_n$ 的重写规则，使得 $A_0, A_1, A_2, \dots, A_n$ 生成终结字符串的过程均不包含栈空状态；则 $A_0, A_1, A_2, \dots, A_n$ 生成终结字符串的状态转移次数必然小于 T 生成终结字符串的状态转移次数

递归的终止条件为单个状态，生成形式为 $A \rightarrow \varepsilon$ 的重写规则

递归过程的反向即为归纳过程，可得到对应的 CFG

因为 NFA 可看作是不对栈进行操作且状态转移不参考栈顶字符的 PDA，所以 NFA 可视为 PDA 的子集，所以正则语言也是上下文无关语言的子集

1.2.3 上下文无关语言的可泵性

定理 上下文无关语言都是可泵的，即：

设 PDA $M = (Q, \Sigma, \Gamma, \delta, q, F)$, $A = L(M)$, $s \in A$ ，则 $\exists p, \forall |s| > p$, s 可表示为 $s = uvxyz$ 且满足： $(|vy| > 0) \wedge (|vxy| \leq p) \wedge (\forall i \in \mathbb{N}, uv^i xy^i z \in A)$

注意：如上可泵性是上下文无关语言的必要条件而非充分条件

上下文无关语言可泵性的证明

将 CFG(V, Σ, R, S) 从初始字符 S 重写到字符串 w 的过程视为生成树，且每个内结点重写为它的所有子节点构成的串

由于每条重写规则重写后得到的字符数是有限的，设单个非终结符最多重写为 b 个字符，则生成树中每个结点最多有 b 个子结点

设生成树高度为 h ，则最终的字符串长度 $|w| \leq b^h$ ，令 $h \geq |V| + 1$ ，则必然存在根结点到叶结点的路径 p 满足 $|p| \geq |V| + 1$ ，在此路径上必然存在重复使用的非终结符

对于该 CFL 中的串 s ，令 $|s| = b^{|V|+2}$ ，找到从根结点到叶结点的最长路径，其中必然包含重复出现的字符 A ；设以距根结点较近的 A 为根的子树为 T_1 ，以距根结点较远的 A 为根的子树为 T_2 ，则 $T_2 \subset T_1$ ；再设 S 生成了 $uvxyz$ ，设 T_1 表示 A 生成了 uvy ， T_2 表示 A 生成了 v ，则用 T_2 整体替换 T_1 可实现 S 生成 uv^0xy^0z ，其它类似，即证

例 证明： $A = \{0^n 1^n 2^n | n \geq 0\}$ 不是上下文无关语言

解 $0^p 1^p 2^p \in A, \because |0^p 1^p 2^p| = 3p > p, \therefore$ 泵引理中 vxy 只可能是 $0^p, 1^p, 2^p, 0^p 1^p, 1^p 2^p$ 其中之一子串, 而无论如何 $uv^0 xy^0 z$ 必然使该串的长度减少, 且不能使 0, 1, 2 的个数同时减少, 所以该语言不可泵, 不是 CFL

例 证明: $A = \{ww | w \in \{0, 1\}^*\}$ 不是上下文无关语言

解 $0^p 1^p 0^p 1^p \in A, \because |0^p 1^p 0^p 1^p| = 4p > p$

- 若泵引理中 vxy 是前半或后半 $0^p 1^p$ 的子串, 则 $uv^0 xy^0 z$ 只能使该串的前半或后半长度减少, 前半和后半不再相同, 所以不可泵
 - 若泵引理中 vxy 是中间 $1^p 0^p$ 的子串, 则 $uv^0 xy^0 z$ 只能使该串前半 1 的个数或后半 0 的个数减少, 前半和后半不再相同, 所以不可泵
- 综上所述, 该语言不满足泵引理, 所以不是 CFL

1.2.4 上下文无关完备的语言运算

定理 若 A, B 都是上下文无关语言, 则 $A \cup B, A \circ B, A^*$ 都是上下文无关语言

上下文无关完备性证明

设 CFG $G_1 = (V_1, \Sigma, R_1, S_1)$ 生成 A , CFG $G_2 = (V_2, \Sigma, R_2, S_2)$ 生成 B

证明: $A \cup B$ 是上下文无关语言

CFG (V, Σ, R, S) 生成 $A \cup B$, 其中 $V = V_1 \cup V_2 \cup \{S\}, R = R_1 \cup R_2 \cup \{S \rightarrow S_1, S \rightarrow S_2\}$

证明: $A \circ B$ 是上下文无关语言

CFG (V, Σ, R, S) 生成 $A \circ B$, 其中 $V = V_1 \cup V_2 \cup \{S\}, R = R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}$

证明: A^* 是上下文无关语言

CFG (V, Σ, R, S) 生成 A^* , 其中 $V = V_1 \cup \{S\}, R = R_1 \cup \{S \rightarrow SS_1, S \rightarrow \varepsilon\}$

而上下文无关语言在交运算和补运算下是不封闭的, 例如 $\{0^n 1^n 2^m | n \geq 0, m \geq 0\}$ 和 $\{0^m 1^n 2^n | n \geq 0, m \geq 0\}$ 都是 CFL, 但二者交集 $\{0^n 1^n 2^n | n \geq 0\}$ 不是 CFL; 同时, 因为 $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$, 若上下文无关语言在补操作下封闭, 则其在交操作下也封闭, 矛盾, 由反证法得出上下文无关语言在补操作下不封闭

定理 若 A 是上下文无关语言, B 是正则语言, 则 $A \cap B$ 是上下文无关语言

上下文无关语言与正则语言交集是上下文无关语言证明

设 PDA $M_1 = (Q_1, \Sigma, \Gamma, \delta_1, s_1, F_1)$ 识别 A , DFA $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ 识别 B

构造 $PDA(Q_1 \times Q_2, \Sigma, \Gamma, \delta, \langle s_1, s_2 \rangle, F_1 \times F_2)$ 识别 $A \cup B$, 其中

$$\delta(q, a) = \begin{cases} (\langle q_1, q_2 \rangle, r') \in \delta(\langle p_1, p_2 \rangle, a, r) & (q_1, r') \in \delta_1(p_1, a, r) \wedge q_2 = \delta_2(p_2, a) \\ (\langle q_1, p_2 \rangle, r') \in \delta(\langle p_1, p_2 \rangle, \varepsilon, r) & (q_1, r') \in \delta_1(p_1, \varepsilon, r) \end{cases}$$

1.2.5 上下文无关语言与可计算理论

给定有限状态机模型, 可解决如下 2 类问题:

- 可接受问题: 某个字符串是否属于某个 CFL

只将该字符串输入到下推自动机中是不可行的, 因为 PDA 允许类似 $(q, \varepsilon) \in \delta(q, \varepsilon, \varepsilon)$ 的转移关系, 下推自动机可能会无限循环

解决方法是将 CFG 转化为乔姆斯基范式, 长度为 $|w|$ 的串必然经历 $2|w| - 1$ 次重写, 而由于重写规则有限, 所以这是可枚举的

- 判空问题: 某个 CFG 对应的语言是否为空, 可利用图遍历算法, 将状态转移函数构造为图, 并搜索初状态到接受状态的路径, 此时可利用 PDA 解决

但不存在可判定等价问题的算法, 即不存在算法判定给定的多个 CFL 是否等价

1.2.6 (*) 确定下推自动机

确定下推自动机 (DPDA = Deterministic Pushdown Automata) 是 6 元组 $(Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q 为机器状态的有限集
- Σ 为可能输入字符的有限集
- Γ 为栈中可能字符的有限集
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \rightarrow P(Q \times (\Gamma \cup \{\varepsilon\}))$ 为状态转移函数, 且 $\forall q \in Q, a \in \Sigma, x \in \Gamma : \delta(q, a, x), \delta(q, a, \varepsilon), \delta(q, \varepsilon, x), \delta(q, \varepsilon, \varepsilon)$ 有且仅有 1 个非空, 且非空的集合只有 1 个元素, 即 DPDA 的转移函数是确定的
- $q_0 \in Q$ 为机器的初状态
- $F \subseteq Q$ 为接受状态集

定理 存在算法判定给定的 DPDA 是否相等, 且算法不存在时间界 ($\forall f(n), \mathbb{P} = \Omega(f(n))$)

2 可计算理论

2.1 丘奇图灵论题

2.1.1 图灵机模型

直观而言，图灵机拥有单向无限长的条带（以单元格为单位，模拟存储），按照状态转移关系，根据指针指向的内容，修改条带内容并移动指针；且机器有 2 个特殊状态：接受状态与拒绝状态，一旦机器进入接受状态或拒绝状态则立即停止，并给出运行结果

图灵机 (TM = (Deterministic) Turing Machine) 是 7 元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$

- Q 为机器状态的有限集
- Σ 为可能输入字符的有限集
- Γ 为条带上可能字符的有限集
- $\delta : Q \times \Sigma \rightarrow Q \times \Gamma \times \{L, R\}$ 为状态转移函数，其中 L, R 表示指针向左或向右移动
- $q_0 \in Q$ 为机器的初状态
- q_{ac} 为机器的接受状态
- q_{rj} 为机器的拒绝状态

特别地：

- 认为 TM 的条带可向右无限延伸
- TM 的输入初始位于条带左侧，指针初始位于条带最左侧
- 允许引入特殊字符 $\sqcup$ ，表示该单元内容为空，且 $\sqcup \notin \Sigma$
- 当指针位于条带最左侧时，再令指针向左移动，则指针忽略该操作
- 图灵机的状态转移关系是确定的，因此对于特定输入的运行结果也是唯一的

格局 (Configuration) 是图灵机某个时刻的运行状态，包含 3 部分：当前状态、指针位置、条带内容，可表示为 uqv ，其中条带内容为 u, v ，当前状态为 q ，指针指向 v 最左侧单元格

特别地，所有 TM 的初始格局均为 q_0w ，其中 w 是输入；接受格局满足 $uq_{ac}v$ 的形式，拒绝格局满足 $uq_{rj}v$ 的形式；称接受格局和拒绝格局为停机格局 (Halting Configuration)

接受与拒绝： 设 M 为 TM， w 为输入字符串， M 以 w 为输入时不断产生新的格局，

- 若最终能够到达接受状态 q_{ac} ，则称 M 接受 (accepts) w
- 若最终能够到达拒绝状态 q_{rj} ，则称 M 拒绝 (rejects) w

注意：图灵机在接受某些输入时可能出现不能终止的情况

识别：若 $A = \{w | M \text{ accepts } w\}$ ，称 M 识别 (recognizes) A ，记作 $A = L(M)$

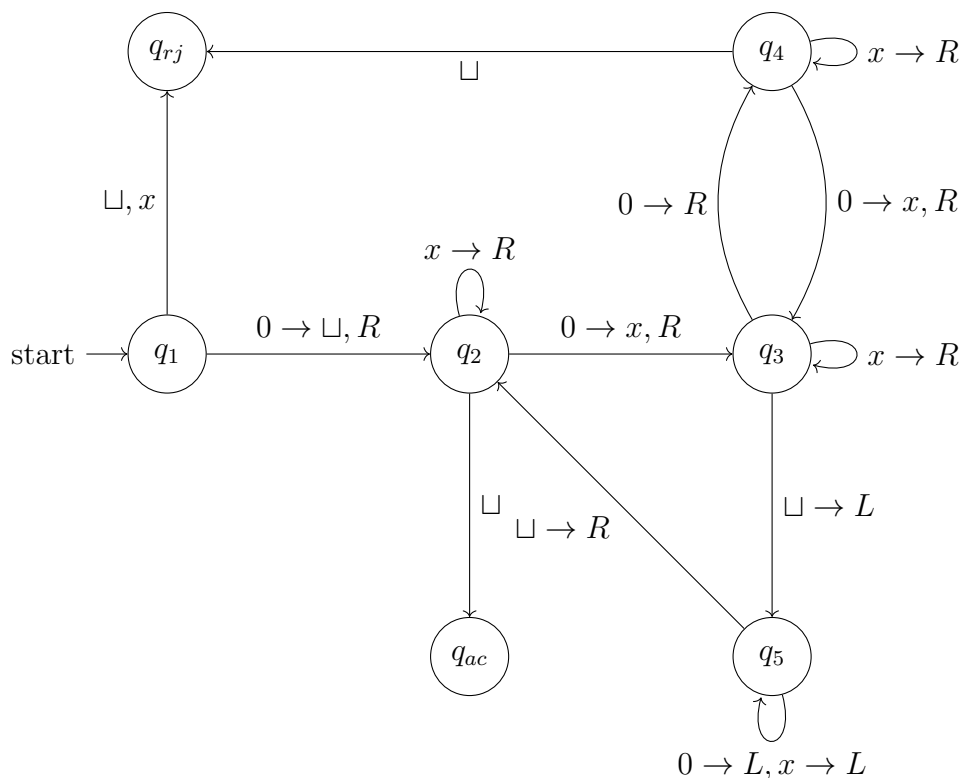
可识别：若 $\exists \text{ TM } M, S = L(M)$ ，则称语言 S 是（图灵）可识别的 (Turing-recognizable)

判定：若 $\exists \text{ TM } M, A = \{w | M \text{ accepts } w\}$ 且 $\bar{A} = \{w | M \text{ rejects } w\}$ ，即： M 以 Σ_A^* 中的字符串为输入时总是能停止，则称 M 判定 (decides) A ，称 A 是可判定的 (Decidable)

例 构造可识别 $A = \{0^{2^n} | n \geq 0\}$ 的图灵机

解 思路：每次从左往右扫描并去掉一半的 0；最终若 0 的个数为 1，则接受；若中途出现了非 0 字符或 0 的个数为奇数，则拒绝

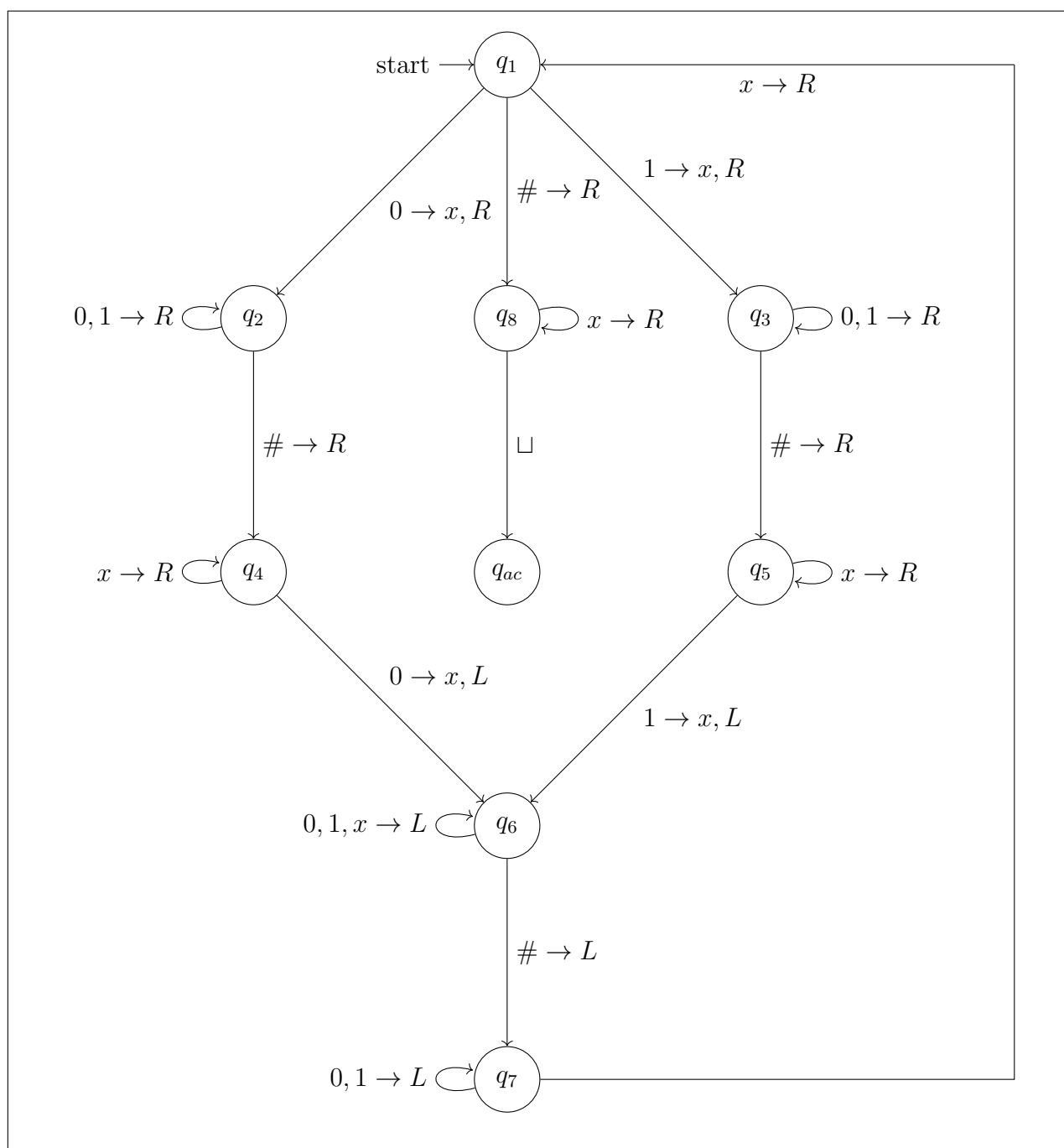
其中，第 1 个 0 会被替换为空字符 $\sqcup$ ，以在指针到达条带最左端时通知机器（实质仍然是当作 0 看待）；其它的 0 会被替换为 x ，表示删除，设计如下：



$q_1 \rightarrow q_2$ 表示将第 1 个 0 变为 $\sqcup$ ，以在指针到达条带最左端时通知机器；第 i 轮扫描过程中， $q_2 \rightarrow q_3$ 表示删除下标为 2^i 位置的 0， $q_3 \rightarrow q_4$ 表示跳过下标为 $k2^i + 2^{i-1}$ 位置的 0， $q_4 \rightarrow q_3$ 表示删除下标为 $k2^i$ 位置的 0；位于 q_4 时，若后面没有 0，表示本轮 0 的个数为奇数，则拒绝；位于 q_3 时，若后面没有 0，表示本轮 0 的个数为偶数，将指针进行左移；最后，位于 q_2 时，若右侧已没有 0，说明只剩第 1 个单元格内的 0，则接受

例 构造可识别 $A = \{w\#w | w \in \{0,1\}^*\}$ 的图灵机

解 思路：每轮在前半部分读取 1 个字符，改为无关字符，并去后半部分匹配字符，也改为无关字符；最后当前半部分匹配完成时，若后半部分匹配完成则接受，构造如下：



2.1.2 变种图灵机

多带图灵机 (MTM = Multiple-tape Turing Machine) 是 7 元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$, 可用多个条带作为存储, 且条带的指针相互独立; 初始时, 输入存放在第 1 条带; 转移函数的形式为 $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$

定理 多带图灵机的能力与单带图灵机相同

多带图灵机与单带图灵机等价证明

可将多带图灵机每条带的内容写到单带上 $\#u_1v_1w_1\#u_2v_2w_2\#u_3v_3w_3\#\dots$ ，其中 $u_iv_iw_i$ 为多带图灵机的各条带内容， v_i 表示多带图灵机第 i 条带上原指针指向字符 v_i ，在此做标记以识别指针

单带图灵机模拟多带图灵机第 i 条带的行为时，首先通过 $\#$ 确定要调整的条带，之后从对应区域寻找带标记的字符，并根据多带图灵机中条带的行为对单带进行调整（若 2 个 $\#$ 之间空间不足，则将右侧 $\#$ 及之后的内容向右移动 1 格，以获得对应条带的空间，模拟多带图灵机的任意条带无限长）

本质上是从并行到串行的转化，时间增加，但能力不变

非确定图灵机 (NTM = Multiple-tape Turing Machine) 是 7 元组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$ ，其中转移关系是非确定的，状态转移函数可写为 $\delta: Q \times \Sigma \rightarrow P(Q \times \Gamma \times \{L, R\})$

多带图灵机的计算过程可视为生成树，其中结点表示格局，边表示状态转移

接受与拒绝： 设 M 为 NTM， M 以 w 为输入时不断产生新的格局（生成树向下延伸），

- 若任意结点最终能够到达接受状态 q_{ac} ，则称 M 接受 (accepts) w
- 若所有分支最终能够到达拒绝状态 q_{rj} ，则称 M 拒绝 (rejects) w

判定： 若 $\exists$ NTM $M, A = \{w | M \text{ accepts } w\}$ 且 $\bar{A} = \{w | M \text{ rejects } w\}$ ，即： M 以 Σ_A^* 中的字符串为输入时总是能停止，则称 M 判定 (decides) A

定理 非确定图灵机的能力与确定图灵机相同

非确定图灵机与确定图灵机等价证明

思路：需要用确定图灵机对非确定图灵机的格局生成树进行搜索；由于该树可能无限生长，所以借鉴 BFS 的思路而非 DFS

设有 3 带图灵机，第 1 条带存放输入，第 2 条带的第 i 单元格存放 n 表示 NTM 第 i 次状态转移选择了映射集合中第 n 个元素（表示格局树 BFS 选择了第 n 个子结点），第 3 条带用于模拟 NTM 的条带

首先确定模拟的状态转移次数 k ，对前 k 次状态转移的可能路径进行枚举，每枚举到 1 个状态转移序列，就将输入复制到第 3 条带上，并根据状态转移序列模拟 NTM 的行为，直到任意转移到达了接受状态，则接受

具体过程：

1. 初始化条带 1 为输入，条带 2 和 3 为空
2. 将条带 1 的内容粘贴到条带 3，并使用条带 3 模拟 NTM 前 k (初始为 1，逐次递增 1) 步的行为；
3. 首次模拟前 k 步的行为时，优先选择 $\delta(q, a)$ 集合中第 1 个元素，并在条带 2 上进行记录；之后模拟前 k 步的行为时，按照字典序更新条带 2 上的字符串为按照字典序

典序的下个字符串，并以此选择路径进行搜索

4. 若任意模拟进入接受状态，则 MTM 接受；若 $\exists k$ ，模拟前 k 步的任意分支均进入了拒绝状态，则 MTM 拒绝

正确性：

- NTM 接受 w 时，MTM 必然能枚举到接受 w 的状态转移
- NTM 某分支拒绝 w 时，MTM 会在此时被阻断，进行下个可能路径的搜索
- NTM 不接受 w 且某分支无限延伸时，MTM 也会无限枚举可能的路径并无限搜索
- NTM 拒绝 w 时，MTM 会在枚举完最长的拒绝路径后也进入拒绝状态

引理 非确定图灵机识别的语言也是可识别的语言

枚举器 (Enumerator) 是可自动产生字符串，可模拟图灵机的行为，可输出字符串的模型

定理 语言 A 是可识别的，当且仅当存在枚举器能够输出 A 的所有字符串²

枚举器可输出可识别语言证明

将 Σ^* 中的字符串按字典序作为横坐标，模拟步数为纵坐标，建立坐标系，并绘制为方格纸，从原点开始进行 BFS，每遍历到 1 个点就进行对应的模拟

由于被接受的字符串 w 从输入到被接受需要模拟的步数是有限的（设为 n 步），则枚举器必然能在 (w, n) 点输出 w

2.1.3 算法的理论基础

丘奇图灵论题：所有“直觉可解决”的问题是图灵机模型可识别的

希尔伯特第十问题：整系数多元整式方程是否有整数根？

希尔伯特第十问题在丘奇图灵论题下是不可判定的，但是可识别的

²本定理一种的错误证明方式及其错误原因参见 Homework 5–Problem 1

2.2 可判定性

2.2.1 正则语言与可判定性

$A_{\text{DFA}} = \{\langle D, w \rangle \mid D \text{ is DFA}, w \in L(D)\}$ 是可判定的

令 TM $M \langle D, w \rangle$:

1. 验证 $\langle D, w \rangle$ 的编码是否合法，若不合法，则 M 拒绝 $\langle D, w \rangle$
2. 根据 D 的状态转移函数和 w 模拟 D 的状态转移
3. 当 w 结束时，若 D 处于接受状态，则 M 接受 $\langle D, w \rangle$ ，否则 M 拒绝 $\langle D, w \rangle$

$A_{\text{NFA}} = \{\langle D, w \rangle \mid D \text{ is DFA}, w \in L(D)\}$ 是可判定的

可将 NFA 转化为 DFA 进行判定，也可用 NTM 直接模拟 NFA

$A_{\text{REG}} = \{\langle R, w \rangle \mid R \text{ is regular expression}, R \text{ generates } w\}$ 是可判定的

将正则表达式转化为 NFA 即可判定

$E_{\text{DFA}} = \{\langle D \rangle \mid D \text{ is DFA}, L(D) = \emptyset\}$ 是可判定的

从初始状态开始对能够到达的状态进行遍历，检查遍历到的状态集是否包含接受状态

$EQ_{\text{DFA}} = \{\langle A, B \rangle \mid A, B \text{ is DFA}, L(A) = L(B)\}$ 是可判定的

根据输入，构造识别 $(L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$ 的 DFA，并执行判空算法即可

2.2.2 上下文无关语言与可判定性

$A_{\text{CFG}} = \{\langle G, w \rangle \mid G \text{ is CFG}, G \text{ generates } w\}$ 是可判定的

不能直接用 NTM 模拟 CFG 的运行，因为 CFG 可能自循环并生成无限长的串

可将 CFG 转化为乔姆斯基范式，此时只需枚举 $2^{|w|} - 1$ 次派生可生成的字符串即可

$E_{\text{CFG}} = \{\langle G \rangle \mid G \text{ is CFG}, L(G) = \emptyset\}$ 是可判定的

循环遍历所有重写规则：若某个字符能够重写为终结符与“可停止的”字符组成的字符串，则标记该字符是可停止的；直到某次遍历后没有新字符被标记为“可停止的”字符，此时检查初始字符是否为可停止的即可；由于重写规则的数量有限，所以可判定

$EQ_{\text{CFG}} = \{\langle G, H \rangle \mid G, H \text{ is CFG}, L(G) = L(H)\}$ 是不可判定的

$A_{\text{CFG}} = \{\langle P, w \rangle \mid P \text{ is PDA}, w \in L(P)\}$ 是可判定的

同样不能直接用 NTM 模拟 PDA 的行为

可将 PDA 转化为 CFG 再转化为乔姆斯基范式进行判定

总结 正则语言 $\subseteq$ 上下文无关语言 $\subseteq$ 可判定语言 $\subseteq$ 可识别语言

2.2.3 不可判定性

$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is TM}, w \in L(M)\}$ 可识别，但不可判定

A_{TM} 可识别证明

图灵机模型 $(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$ 是有限的 7 元组，所以图灵机能够模拟其它图灵机的运行（编译器的数学基础），当 M 接受 w 时必然能够得到运行结果

 A_{TM} 不可判定证明

从势的角度为以下的证明做铺垫：图灵机本质 $(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$ 是有限的 7 元组，可转化为二进制编码，因此与自然数集等势；不同图灵机识别的语言可能相同，所以可识别语言的数量不多于图灵机的数量；语言可视为字符串集的子集，因此语言的数量与字符串集的幂集等势；又因为字符串可用二进制编码，与自然数集等势，所以语言集合的势高于图灵机集合的势，必然存在不可识别的语言

反证法证明：假设 A_{TM} 是可判定的，设 H 能够判定 A_{TM}

令 TM $D\langle M \rangle$:

1. 模拟 $H\langle M, M \rangle$, i.e. 利用 H 判定 $M\langle M \rangle$
2. 若 H 接受 $\langle M, M \rangle$, 则 G 拒绝 $\langle M \rangle$
3. 若 H 拒绝 $\langle M, M \rangle$, 则 G 接受 $\langle M \rangle$

则对于 $D\langle D \rangle$, 首先 H 判定 $D\langle D \rangle$, 若 D 接受 D , 则 D 拒绝 D , 显然矛盾
矛盾的产生只可能由于判定 A_{TM} 的 H 是不存在的, 即 A_{TM} 不可判定

补可识别: 若 $\bar{A} = \Sigma^* - A$ 是可识别的, 称 A 是补可识别的 (co-Turing-recognizable)

定理 语言 A 是可判定的, 当且仅当 A 和 $\bar{A}$ 都是可识别的

可判定 $\Rightarrow$ 原语言和补语言可识别证明

设 $TMM(Q, \Sigma, \Gamma, \delta, q_0, q_{ac}, q_{rj})$ 可判定 A , 则 M 必然可识别 A ; 交换 q_{ac} 和 q_{rj} 后, 新的图灵机可判定 $\bar{A}$, 则 $\bar{A}$ 也是可识别的

原语言和补语言可识别 $\Rightarrow$ 可判定证明

设 TMM_1, M_2 分别可识别 $A, \bar{A}, w \in \Sigma_A^*$, 令 TM $M\langle w \rangle$:

1. 交替模拟 $M_1\langle w \rangle$ 和 $M_2\langle w \rangle$, 直到某个运行结果为接受
2. 若 M_1 接受 w , 则 M 接受 w
3. 若 M_2 接受 w , 则 M 拒绝 w

由于 $w \in A$ 和 $w \in \bar{A}$ 必然有且仅有 1 个成立, 所以以上过程必然能够停止, 满足可判定的定义, 即证 A 可判定

根据上述定理, 利用反证法, 可得出 $\overline{A_{TM}}$ 是不可识别的

2.3 归约

2.3.1 其它不可判定的语言

$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ loops on } w\}$ 是不可判定的

$HALT_{TM}$ 不可判定性证明

归约+反证法证明：假设 TM H 判定 $HALT_{TM}$ ，则有 $H\langle M, w \rangle$:

1. 若 M 接受 w 或 M 拒绝 w ，则 H 接受 $\langle M, w \rangle$
2. 若 $M\langle w \rangle$ 无限循环，则 H 拒绝 $\langle M, w \rangle$

构造 TM B ，令 $B\langle M, w \rangle$:

1. 运行 $H\langle M, w \rangle$ ，则由可判定性，运行结果只可能是接受或拒绝
2. 若 H 拒绝 $\langle M, w \rangle$ ，则 B 拒绝 $\langle M, w \rangle$
3. 若 H 接受 $\langle M, w \rangle$ ，则运行 $M\langle w \rangle$ ，此时运行结果只可能是接受或拒绝；
4. 若 M 接受 w ，则 B 接受 $\langle M, w \rangle$
5. 若 M 拒绝 w ，则 B 拒绝 $\langle M, w \rangle$

则 B 可作为 A_{TM} 问题的判定器，与 A_{TM} 的不可判定性矛盾，因此 H 不存在，所以 $HALT_{TM}$ 不可判定

$E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$ 是不可判定的

E_{TM} 不可判定性证明

归约+反证法证明：假设 TM E 判定 E_{TM} ，则有 $E\langle M \rangle$:

1. 若 $L(M) = \emptyset$ ，则 E 接受 $\langle M \rangle$
2. 若 $L(M) \neq \emptyset$ ，则 E 拒绝 $\langle M \rangle$

构造 TM $N_{\langle M, w \rangle}$ ，令 $N_{\langle M, w \rangle}\langle x \rangle$:

1. 若 $x \neq w$ ，则 $N_{\langle M, w \rangle}$ 拒绝 x
2. 若 $x = w$ ，则运行 $M\langle w \rangle$
3. 若 M 接受 w ，则 $N_{\langle M, w \rangle}$ 接受 x
4. 若 M 拒绝 w ，则 $N_{\langle M, w \rangle}$ 拒绝 x

设 $\langle M, w \rangle \in \Sigma_{A_{TM}}^*$ ，则 $N_{\langle M, w \rangle} \in E_{TM} \Leftrightarrow \langle M, w \rangle \notin A_{TM}$ ，此时 $\overline{A_{TM}}$ 可判定，则 A_{TM} 也可判定，与 A_{TM} 的不可判定性矛盾，因此 E 不存在，所以 E_{TM} 不可判定

$REG_{TM} = \{\langle M \rangle \mid L(M) \text{ is regular language}\}$ 是不可判定的

REG_{TM} 不可判定性证明

归约+反证法证明：假设 $TM R$ 判定 REG_{TM} ，则有 $R\langle M \rangle$ ：

1. 若 $L(M)$ 是正则语言，则 R 接受 $\langle M \rangle$
2. 若 $L(M)$ 不是正则语言，则 R 拒绝 $\langle M \rangle$

构造 $TM N_{\langle M, w \rangle}$ ，令 $N_{\langle M, w \rangle}\langle x \rangle$ ：

1. 若 $x \in \{0^n 1^n | n \geq 0\}$ ，则 $N_{\langle M, w \rangle}$ 接受 x
2. 若 $x \notin \{0^n 1^n | n \geq 0\}$ ，则运行 $M\langle w \rangle$
3. 若 M 接受 w ，则 $N_{\langle M, w \rangle}$ 接受 x
4. 若 M 拒绝 w ，则 $N_{\langle M, w \rangle}$ 拒绝 x

因为 $\{0^n 1^n\}$ 不是正则语言，有：

- M 接受 $w \Leftrightarrow L(N_{\langle M, w \rangle}) = \{0, 1\}^* \Leftrightarrow L(N_{\langle M, w \rangle})$ 是正则语言 $\Leftrightarrow \langle N_{\langle M, w \rangle} \rangle \in L(R)$
- M 拒绝 $w \Leftrightarrow L(N_{\langle M, w \rangle}) = \{0^n 1^n\} \Leftrightarrow L(N_{\langle M, w \rangle})$ 不是正则语言 $\Leftrightarrow \langle N_{\langle M, w \rangle} \rangle \notin L(R)$

设 $\langle M, w \rangle \in \Sigma_{A_{TM}}^*$ ，则 $N_{\langle M, w \rangle} \in REG_{TM} \Leftrightarrow \langle M, w \rangle \in A_{TM}$ ，此时 A_{TM} 可判定，与 A_{TM} 的不可判定性矛盾，因此 R 不存在，所以 REG_{TM} 不可判定

$EQ_{TM} = \{\langle M_1, M_2 \rangle | L(M_1) = L(M_2)\}$ 是不可判定的

 EQ_{TM} 不可判定性证明

令 M_2 满足 $L(M_2) = \emptyset$ ，若 EQ_{TM} 可判定，则判定 $\langle M_1, M_2 \rangle \in EQ_{TM}$ 的真假即可判定 $\langle M_1 \rangle \in E_{TM}$ 的真假，与 E_{TM} 的不可判定性矛盾，因此 EQ_{TM} 不可判定

2.3.2 涉及计算历史的归约

计算历史： 设 $TM M, w \in \Sigma_M^*$ ，则 $M\langle w \rangle$ 运行过程中 M 的所有格局的序列 $C_1 C_2 C_3 \cdots C_k$ (k 为常数，或 $k \rightarrow \infty$) 称为 $M\langle w \rangle$ 的计算历史

线性有界自动机 (LBA = Linear bounded automata) 是条带长度有限的图灵机

默认条带长度为 $|w|$ ，实际应用时条带长度可为 $O(|w|)$

$A_{LBA} = \{\langle M, w \rangle | M \text{ is LBA, } M \text{ accepts } w\}$ 是可判定的

 A_{LBA} 可判定性证明

因为 M 的状态数 $|Q|$ ，条带字符集基数 $|\Gamma|$ ，条带长度 $|w|$ 都是有限的，不同的格局数量最多只有 $O(|w||Q||\Gamma|^{|w|})$ 种，是有限的

若模拟 $M\langle w \rangle$ 的步数超出了该界，则必然陷入循环，可拒绝，因此 A_{LBA} 可判定

$E_{LBA} = \{\langle M, w \rangle | M \text{ is LBA, } L(M) = \emptyset\}$ 是不可判定的

E_{LBA} 不可判定性证明

归约+反证法证明：假设 TM E 判定 E_{LBA} ，则有 $E\langle M \rangle$ ：

1. 若 $L(M) = \emptyset$ ，则 E 接受 $\langle M \rangle$
2. 若 $L(M) \neq \emptyset$ ，则 E 拒绝 $\langle M \rangle$

构造 TM B ，其中 $B\langle x \rangle$ 将 x 拆分为 $M\langle w \rangle$ 运行过程中的格局序列 $C_1 C_2 C_3 \cdots C_l$ ，并依次验证如下命题是否为真：

1. C_1 是 $M\langle w \rangle$ 的初始格局
2. $C_i \rightarrow C_{i+1}$ 符合 M 的转移规则
3. C_l 是 M 的接受格局

由于可拆分的组合有限，且最多需要 $O(|w|)$ 的空间验证以上命题的真假，所以 B 可构造为 LBA 以作为 E 的输入，此时 B 接受 x 当且仅当 x 为 M 接受 w 的格局序列，即 $L(B) \notin E_{LBA}$ 当且仅当 $\langle M, w \rangle \in A_{TM}$ ，所以 E_{LBA} 可判定时 A_{TM} 亦可判定，与 A_{TM} 的不可判定性矛盾，因此 E_{LBA} 不可判定

$ALL_{CFG} = \{\langle G \rangle \mid G \text{ is CFG}, L(G) = \Sigma_G^*\}$ 是不可判定的

 ALL_{CFG} 不可判定性证明

归约+反证法证明：假设 CFG G 生成 TM M 的格局序列（计算历史），且该格局序列不是 $M\langle w \rangle$ 运行到接受状态的格局序列，即 G 产生的 $C_1 \# C_2 \# C_3 \# \cdots \# C_l$ 满足：

- C_1 不是 $M\langle w \rangle$ 的初始格局
- $C_i \rightarrow C_{i+1}$ 不符合 M 的转移规则
- C_l 不是 M 的接受格局

具体而言，构造 PDA D ，根据 PDA 的运行原理， D 以 $C_1 \# C_2^R \# C_3 \# C_4^R \# \cdots$ 的形式生成非 $M\langle w \rangle$ 的格局序列，状态转移关系设计为到达 PDA 的接受状态时满足以上 3 条性质中任意 1 条性质，即可根据 $M\langle w \rangle$ 得到对应的 G ，则此时有：

- 若 $\langle M, w \rangle \in A_{TM}$ ，即 $w \in L(M)$ ，则 $L(G) \neq \Sigma_G^*$
- 若 $\langle M, w \rangle \notin A_{TM}$ ，即 $w \notin L(M)$ ，则 $L(G) = \Sigma_G^*$

所以 ALL_{CFG} 可判定时 $\overline{A_{TM}}$ 也可判定，对应 A_{TM} 可判定，这与 A_{TM} 的不可判定性矛盾，因此 ALL_{CFG} 不可判定

2.3.3 映射归约

可计算函数：设双带图灵机 M ，若函数 f 的定义域为 $D(f)$ ，且 $\forall w \in D(f), M\langle w \rangle$ 均能终止，且在第 2 条带得到 $f(w)$ ，则称 f 为可计算函数

映射归约: 设语言 A, B , 若存在可计算函数 $f: \Sigma_A^* \rightarrow \Sigma_B^*$ 使得 $\forall w \in \Sigma_A^*, w \in A \Leftrightarrow f(w) \in B$, 称 A 可映射归约至 B , 记作 $A \leq_m B$

映射归约有如下性质:

- $A \leq_m A$ (构造 $f(w) = w$ 显然 f 可计算)
- $A \leq_m B \wedge B \leq_m C \Rightarrow A \leq_m C$ (f, g 均可计算则显然 $g \circ f$ 可计算)
- 若 $A \leq_m B$, 且 B 是可判定的, 则 A 是可判定的 (可运行 f 后运行 B 的判定算法)
- 若 $A \leq_m B$, 且 A 是不可判定的, 则 B 是不可判定的
- 若 $A \leq_m B$, 且 B 是可识别的, 则 A 是可识别的
- 若 $A \leq_m B$, 且 A 是不可识别的, 则 B 是不可识别的
- $A \leq_m B \Leftrightarrow \overline{A} \leq_m \overline{B}$

例 证明: $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$

解 设 $\langle M, w \rangle \in \Sigma_{A_{\text{TM}}}^*$, 将 M 的拒绝状态 q_{rj} 设为普通状态 q , 并添加 q 读取任意字符时均向左移动指针并回到状态 q 的状态转移关系 (自循环), 另设不可达的拒绝状态 q'_{rj} , 新得到的图灵机记为 M' , 记以上操作为 $f(\langle M, w \rangle) = \langle M', w \rangle$

则显然 f 可计算, 且 $\langle M, w \rangle \in A_{\text{TM}} \Leftrightarrow f(\langle M, w \rangle) = \langle M', w \rangle \notin \text{HALT}_{\text{TM}}$, 即证

EQ_{TM} 既不是可识别的, 也不是补可识别的

EQ_{TM} 不可识别性证明

归约: $\overline{A_{\text{TM}}} \leq_m EQ_{\text{TM}}$

构造 $f: \Sigma_{A_{\text{TM}}}^* \rightarrow \Sigma_{EQ_{\text{TM}}}^*$, 令 $f(\langle M, w \rangle) = \langle M_{\emptyset}, M_{\langle M, w \rangle} \rangle$

其中 $L(M_{\emptyset}) = \emptyset$, 而 $M_{\langle M, w \rangle} \langle x \rangle$:

1. 模拟 $M \langle w \rangle$
2. 若 M 接受 w , 则 $M_{\langle M, w \rangle}$ 接受 x
3. 若 M 拒绝 w , 则 $M_{\langle M, w \rangle}$ 拒绝 x

此时有:

- $w \in L(M) \Leftrightarrow L(M_{\langle M, w \rangle}) = \Sigma^* \Leftrightarrow L(M_{\emptyset}) \neq L(M_{\langle M, w \rangle})$
- $w \notin L(M) \Leftrightarrow L(M_{\langle M, w \rangle}) = \emptyset \Leftrightarrow L(M_{\emptyset}) = L(M_{\langle M, w \rangle})$

说明: f 是将 M, w 的信息置于 $M_{\langle M, w \rangle}$ 上, 并将 $M_{\langle M, w \rangle}$ 输出, 因此 f 的可计算性与 $M \langle w \rangle$ 的运行情况是无关的, 所以 f 可计算, 有 $A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$

根据映射归约的性质, $\overline{A_{\text{TM}}} \leq_m EQ_{\text{TM}}$, 由于 A_{TM} 不可识别, 所以 EQ_{TM} 不可识别

$\overline{EQ_{\text{TM}}}$ 不可识别性证明

归约: $\overline{A_{\text{TM}}} \leq_m \overline{EQ_{\text{TM}}}$

将上述 $f(\langle M, w \rangle) = \langle M_\emptyset, M_{\langle M, w \rangle} \rangle$ 改为 $g(\langle M, w \rangle) = \langle M_\Sigma, M_{\langle M, w \rangle} \rangle$, 其中 $L(M_\Sigma) = \Sigma^*$ 可证明 $A_{\text{TM}} \leq_m EQ_{\text{TM}}$, 再根据映射归约的性质即证

Rice 定理 设 P 是图灵机模型的集合, 若 P 满足:

- P 是非奇异的, 即 $P \neq \emptyset \wedge P \neq \Sigma^*$;
- 若 $L(M_1) = L(M_2)$, 则 $\langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P$;

则 P 是不可判定的

证明过程请参考 Homework 7–Problem 4, 简单应用请参考 Homework 7–Problem 5

3 复杂性理论

3.1 时间复杂度

3.1.1 时间复杂度的表述

时间复杂性的表述：设 TM M ，可能的输入为 Σ^* ，且假设 M 在任意输入上均能停机

- 运行时间函数 $f_1 : \Sigma^* \rightarrow \mathbb{N}$ ，其中 $f_1(x)$ 为 M 从开始运行到停机状态的状态转移次数
- 最坏情况下的时间复杂度 $f_2(n) = \max_{x \in \Sigma^n} f_1(x)$
- 平均情况下的时间复杂度 $f_3(n) = \sum_{x \in \Sigma^n} f_1(x) / |\Sigma|^n$

关于 O, Θ, Ω, o 表示法，请参考 **CS2308算法与复杂性** 课程笔记第 3.2 节

例 分析 TM 判定 $\{0^k 1^k | k \geq 0\}$ 的时间复杂度

解 根据使用的算法不同，时间复杂度也不尽相同

若每次验证 1 个 0 对应 1 个 1，则时间复杂度为 $O(n^2)$

若每次删除一半的 0 和 1，则时间复杂度为 $O(n \log n)$

若用双带图灵机则可将时间复杂度降为 $O(n)$

定理 非正则语言在图灵机的判定时间为 $\Omega(n \log n)$ ，正则语言在 $o(n \log n)$ 时间内可判定

定理 若 $t(n) = \Omega(n)$ ，则在多带图灵机上需要 $O(t(n))$ 时间完成的算法，在 $O(t^2(n))$ 时间内可在单带图灵机上完成

定理 若 $t(n) = \Omega(n)$ ，则在非确定图灵机上需要 $O(t(n))$ 时间完成的算法，在 $O(2^{t(n)})$ 时间内可在确定单带图灵机上完成

3.1.2 P 类

P 类：在 $O(n^k), k \in \mathbb{N}$ 时间内可在单带图灵机上判定的问题

根据上面的推论，P 类等价于在 $O(n^k), k \in \mathbb{N}$ 时间内可在多带图灵机上判定的问题

例 证明 A_{CFG} 可在 $O(|w|^k)$ 时间内判定

解 不可以用乔姆斯基范式枚举，因为需要的枚举次数为 $|\delta|^{2|w|-1}$ ，是 $2^{O(|w|)}$ 时间的可用如下动态规划算法判定，是 $O(|w|^k)$ 时间的：

1. 将 G 转化为乔姆斯基范式
2. 对 $w = w_1 w_2 w_3 \cdots w_n$ ，构造动态规划表 $dp[n][n]$
 $dp[i][j]$ 中的非终结符串能够生成 $w_i \cdots w_j$
 - 对 $j = i$ ，只需验证是否存在重写规则 $X \rightarrow w_i$

• 对 $j - i \geq 1$, 需要验证 $\exists k \in [i, j - 1]$ 时 $dp[i][k]$ 和 $dp[k + 1, j]$ 均能生成
所以可在 $O(|w|^3)$ 时间内判定 w 能否被 G 生成

3.1.3 NP 类

NP 类: 在 $O(n^k), k \in \mathbf{N}$ 时间内可在非确定图灵机上判定的问题

注意: $\mathbf{N}$ 不表示 “Not”, 而表示 “Non-deterministic”, 即 “非确定”

验证器 (Verifier): 若 $w \in A \Rightarrow \exists c, V$ 接受 $\langle w, c \rangle$, 则称 V 是 A 的验证器

定理 若 $w \in A$, V 是 A 的验证器, 且可在 $O(|w|^k)$ 时间内运行 $V\langle w, c \rangle$, 则 $A \in \mathbf{NP}$

TM 多项式时间可验证与 NTM 多项式时间可判定等价证明

1. 假设 V 是 A 的验证器, 则可构造 NTM N , 令 $N\langle w \rangle$:
 - (a) 非确定地生成 c
 - (b) 调用 $V\langle w, c \rangle$ 算法
 即可在多项式时间内运行 $N\langle w \rangle$
2. 假设 NTM N 可在多项式时间内判定 A , 则可在多项式时间内生成 c 并用 V 验证

例 团集问题表述为 $CLIQUE = \{\langle G, k \rangle \mid G \text{ has a complete subgraph of } k \text{ vertices}\}$

证明: $CLIQUE \in \mathbf{NP}$

解 NTM 多项式时间可判定: 随机选择 N 个结点并验证是否是完全子图

TM 多项式时间可验证: 令 $V\langle G, k, c \rangle$, 其中 c 是点集:

1. 若 $|c| \neq k$, 则 V 拒绝 $\langle G, k, c \rangle$
2. 若 $\exists p \in c : p \notin G$, 则 V 拒绝 $\langle G, k, c \rangle$
3. 若 $\exists a \in c, b \in c : a \neq b \wedge \{a, b\} \notin E(G)$, 则 V 拒绝 $\langle G, k, c \rangle$
4. V 接受 $\langle G, k, c \rangle$

3.1.4 NP 完全性

多项式时间归约: 设 A 可映射归约至 B , 且 $f : \Sigma_A^* \rightarrow \Sigma_B^*$ 是可计算函数, 若对于 $\forall w \in \Sigma_A^*$ 有 $f(w)$ 可在多项式时间内完成, 则称 A 可多项式时间归约至 B , 记为 $A \leq_P B$

定理 若 $A \leq_P B$, 且 $B \in \mathbf{P}$, 则 $A \in \mathbf{P}$

例 证明: 3-SAT 问题 (每个子句含有 3 个文字的合取范式可满足问题) 可多项式时间归约至团集问题 (图是否含有 k 个结点的完全子图的问题)

解 将每个文字变化为结点（重复出现的相同变元也变为不同结点），对于不在同个子句，且不是彼此的否的 2 个变元，对应的结点对用边连接

设 3-SAT 问题的对象包含 k 个子句，则：3-SAT 问题中的合取范式可满足，当且仅当对应团集问题的图中包含 k 个结点的团集，即证

NP 难类：若问题 B 满足 $\forall A \in \text{NP}, A \leq_p B$ ，则称 B 是 NP 难的，即 $B \in \text{NP-H}$

NP 完全类：若 A 既是 NP 问题，又是 NP-H 问题，则称 A 是 NP 完全的，即 $A \in \text{NP-C}$

定理 若 $A \leq_p B$ ，且 $B \in \text{NP-C}$ ，则 $A \in \text{NP-C}$

定理 $\text{SAT} \in \text{NP-C}$

SAT $\in$ NP 证明

为合取范式中的变元随机赋值并验证是否满足，可在多项式时间内验证 SAT 问题

SAT $\in$ NP-C 证明

对于 $A \in \text{NP}$ ，存在 $N \in \text{NTM}$ 可在 $O(n^k)$ 时间内判定 A

维护 $n^k \times n^k$ 的表，每行放置 $N\langle x \rangle$ 的格局，每个格子放 1 个字符

引入布尔变量 $x_{i,j,s}$ 表示第 i 行第 j 列的字符是否为 s ，设 φ_1 表示每个格子的内容合法，即有且仅有 1 个 $Q \cup \Gamma \cup \{\sqcup\}$ 中的字符； φ_2 表示第 1 行为 $N\langle x \rangle$ 的初始格局； φ_3 表示出现 $N\langle x \rangle$ 的接受格局； φ_4 表示从第 i 行到第 $i+1$ 行满足 N 的转移规则

当 N 接受 x 时，应有 $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ ，根据如上描述可写为表达式：

$$\varphi_1 = \bigwedge_{i=1}^{n^k} \bigwedge_{j=1}^{n^k} \left[\left(\bigvee_{s \in Q \cup \Gamma \cup \{\sqcup\}} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, s' \in Q \cup \Gamma \cup \{\sqcup\} \\ s \neq s'}} \overline{x_{i,j,s} \wedge x_{i,j,s'}} \right) \right]$$

$$\varphi_2 = x_{1,1,q_0} \wedge x_{1,2,w_1} \wedge x_{1,3,w_2} \wedge \cdots \wedge x_{1,n+1,w_n} \wedge x_{1,n+2,\sqcup} \wedge x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k,\sqcup}$$

$$\varphi_3 = \bigvee_{i=1}^{n^k} \bigvee_{j=1}^{n^k} x_{i,j,q_{ac}}$$

构造以上 3 个合取范式的时间复杂度分别为 $O(n^{2k}), O(n^k), O(n^k)$ 均为多项式时间

而对于 φ_4 ，可根据窗口平移的方式验证，定义 2 行 3 列的格子组成 1 个窗口，定义布尔变量 $w_{i,j}$ 表示以第 i 行第 j 列的格子为中上格子的窗口的合法性

由于给定的 NTM 中 $|Q|$ 和 $|T|$ 均为常量，不同窗口的种类数不超过 $(|Q| + |T| + 1)^6$ 也是常量，所以构造 $w_{i,j}$ 的真值表只需要常量时间

所以 $\varphi_4 = \bigwedge_{i=2}^{n^k-1} \bigwedge_{j=2}^{n^k-1} w_{i,j}$ 的复杂度也是 $O(n^{2k})$

所以任意 NTM 的计算历史均可多项式时间归约到合取范式，有 $x \in A \Leftrightarrow f(x) \in \text{SAT}$ ，所以 $\text{SAT} \in \text{NP-H}$ ，进而 $\text{SAT} \in \text{NP-C}$

定理 $3-SAT \in NP - C$

$3-SAT \in NP$ 证明

随机为所有变元赋值并验证是否满足，可在多项式时间内验证 $3-SAT$ 问题

$3-SAT \in NP - C$ 证明: $SAT \leq_P 3-SAT$

对于 SAT 问题中的每个子句 f

- 若 f 中的文字数为 1，设 $f = x$ ，引入 y_1, y_2 ，将 f 替换为 $f' = (x \vee y_1 \vee y_2) \wedge (x \vee y_1 \vee \bar{y}_2) \wedge (x \vee \bar{y}_1 \vee y_2) \wedge (x \vee \bar{y}_1 \vee \bar{y}_2)$
- 若 f 中的文字数为 2，设 $f = (x \vee y)$ ，引入 z ，将 f 替换为 $f' = (x \vee y \vee z) \wedge (x \vee y \vee \bar{z})$
- 若 f 中的文字数恰好为 3，则 f 保持不变
- 若 f 中的文字数超过 3，设 $f = (x_1 \vee x_2 \vee \dots \vee x_k), k > 3$ ，引入 $y_1, y_2, \dots, y_{k-3}$ ，将 f 替换为 $f' = (x_1 \vee x_2 \vee y_1) \wedge (\bar{y}_1 \vee x_3 \vee y_2) \wedge (\bar{y}_2 \vee x_4 \vee y_3) \wedge \dots \wedge (\bar{y}_{k-4} \vee x_{k-2} \vee y_{k-3}) \wedge (\bar{y}_{k-3} \vee x_{k-1} \vee x_k)$

对于 SAT 问题中的每个子句 f ，按照上述方式替换后得到的合取范式 f' 中的每个子句均包含 3 个文字，所以得到的合取范式是 $3-SAT$ 问题的对象

由于可在关于 SAT 问题的文字数的多项式时间内完成上述替换，所以 $SAT \leq_P 3-SAT$ ，进而 $3-SAT \in NP - C$

定理 顶点覆盖问题 $VC = \{\langle G, k \rangle \mid G \text{ has a vertex cover of } k \text{ vertices}\} \in NP - C$

$VC \in NP$ 证明

随机选择 k 个结点并验证是否是顶点覆盖，可在多项式时间内验证 VC 问题

$VC \in NP - C$ 证明: $3-SAT \leq_P VC$

对于 $3-SAT$ 问题中的每个子句 $f = (x \vee y \vee z)$ ，构造三角形，其中每个结点分别对应 x, y, z ，对于 $3-SAT$ 问题中出现的每个变元 x ，构造 2 个结点 v_x 和 $v_{\bar{x}}$ ，并用边连接 v_x 和所有子句中对应 x 的结点，用边连接 $v_{\bar{x}}$ 和所有子句中包含 $\bar{x}$ 的结点

设 $3-SAT$ 问题中的合取范式包含 k 个子句，则构造的图中存在 k 个三角形，且每个三角形的结点分别对应一个子句中的 3 个文字；设 $3-SAT$ 问题中的合取范式包含 n 个不同变元，则构造的图中存在 $2n$ 个结点对应变元或其非

当 $3-SAT$ 问题中的合取范式可满足时，存在满足条件的赋值，使得每个子句中至少有 1 个文字为真，则对于每个子句对应的三角形，选择 1 个对应真文字的结点加入顶点覆盖；对于每个变元 x ，若 x 的值为真，则选择 v_x 加入顶点覆盖；若 x 的值为假，则选择 $v_{\bar{x}}$ 加入顶点覆盖；则 $3-SAT$ 问题中的合取范式可满足，当且仅当构造的图中存在 $k + n$ 个结点的顶点覆盖

由于可在关于 $3-SAT$ 问题的子句数和变元数的多项式时间内完成上述构造, 所以 $3-SAT \leq_P VC$, 进而 $VC \in NP-C$

定理 有向图的哈密顿路径是 NP 完全问题

$$H = \{\langle G, s, t \rangle \mid \text{Directed } G \text{ has a Hamiltonian path from } s \text{ to } t\} \in NP-C$$

$H \in NP$ 证明

以 s 为起始结点, 以 t 为结束结点, 随机生成其它结点的序列并验证是否是哈密顿路径, 可在多项式时间内验证 H 问题

$H \in NP-C$ 证明: $3-SAT \leq_P H$

设 $3-SAT$ 问题中的合取范式包含 k 个子句, 对于 $3-SAT$ 问题中的每个变元 x , 构造 $3k+1$ 个结点 $v_{x,1}, v_{x,2}, \dots, v_{x,3k+1}$, 并用双向边连接 $(v_{x,i}, v_{x,i+1}), i = 1, 2, 3, \dots, 3k$

对于 $3-SAT$ 问题中的每个子句 f_i , 构造 1 个结点 c_i ; 若变元 x 在子句 f_i 中以正文字出现在第 $j(1 \leq j \leq 3)$ 个位置, 则用边分别连接 $(v_{x,3(i-1)+j}, v_{c_i})$ 与 $(v_{c_i}, v_{x,3(i-1)+j+1})$; 若变元 x 在子句 f_i 中以负文字出现在第 $j(1 \leq j \leq 3)$ 个位置, 则用边分别连接 $(v_{x,3(i-1)+j+1}, v_{c_i})$ 与 $(v_{c_i}, v_{x,3(i-1)+j})$

设所有变元 $x_1, x_2, x_3, \dots, x_n$, 在 x_a 与 x_{a+1} 之间添加结点 $u_{a,a+1}(0 \leq a \leq n)$, 并用边分别连接 $(v_{x_a,1}, u_{a,a+1}), (v_{x_a,3k+1}, u_{a,a+1}), (u_{a,a+1}, v_{x_{a+1},1}), (u_{a,a+1}, v_{x_{a+1},3k+1})$

当变元 x_a 的值为真时, 哈密顿路径在 $u_{a-1,a}$ 与 $u_{a,a+1}$ 之间的路径为 $u_{a-1,a} \rightarrow v_{x_a,1} \rightarrow v_{x_a,2} \rightarrow \dots \rightarrow v_{x_a,3k+1} \rightarrow u_{a,a+1}$, 过程中可进入包含 x_a 的子句对应的结点 c_i ; 当变元 x_a 的值为假时, 哈密顿路径在 $u_{a-1,a}$ 与 $u_{a,a+1}$ 之间的路径为 $u_{a-1,a} \rightarrow v_{x_a,3k+1} \rightarrow v_{x_a,3k} \rightarrow \dots \rightarrow v_{x_a,1} \rightarrow u_{a,a+1}$, 过程中可进入包含 $\bar{x}_a$ 的子句对应的结点 c_i

注意: 路径不能从 x_a 对应的某个结点 $x_{a,j}$ 进入 c_i 后回到 $x_b(b \neq a)$ 对应的某个结点 $x_{b,l}$, 否则路径无法同时访问到 $x_{a,j}$ 的相邻节点和 $x_{b,l}$ 的相邻结点, 所以哈密顿路径在 $u_{a-1,a}$ 与 $u_{a,a+1}$ 之间的路径只能是上述两种情况之一

此时, c_i 对应的子句 f_i 中至少有 1 个文字为真, 当且仅当哈密顿路径可进入 c_i 并离开 c_i , 否则哈密顿路径无法进入 c_i

最后, 将得到的 $\langle G, s, t \rangle$ 作为 Σ_H^* 的对象, 其中 $s = u_{0,1}, t = u_{n,n+1}$

由于可在关于 $3-SAT$ 问题的子句数和变元数的多项式时间内完成上述构造, 所以 $3-SAT \leq_P H$, 进而 $H \in NP-C$

定理 无向图的哈密顿路径是 NP 完全问题

$$UH = \{\langle G, s, t \rangle \mid \text{Undirected } G \text{ has a Hamiltonian path from } s \text{ to } t\} \in NP-C$$

$UH \in \text{NP}$ 证明

以 s 为起始结点, 以 t 为结束结点, 随机生成其它结点的序列并验证是否是哈密顿路径, 可在多项式时间内验证 UH 问题

 $UH \in \text{NP} - \text{C}$ 证明: $H \leq_P UH$

对于 H 问题中的每个结点 $v(v \neq s, t)$, 构造 3 个结点 v_{in}, v_{mid}, v_{out} , 并用边连接 $(v_{in}, v_{mid}), (v_{mid}, v_{out}), (v_{out}, v_{in})$; 对于 H 问题中的每条边 (u, v) , 用边连接 (u_{out}, v_{in})

对于 s , 构造结点 s_{out} ; 对于 H 问题中的每条边 (s, v) , 用边连接 (s_{out}, v_{in}) ; 对于 t , 构造结点 t_{in} ; 对于 H 问题中的每条边 (u, t) , 用边连接 (u_{out}, t_{in})

当 H 问题中的图 G 中存在从 s 到 t 的哈密顿路径

$$s, v_1, v_2, \dots, v_{n-2}, t$$

时, UH 问题中的图 G' 中存在从 s_{out} 到 t_{in} 的哈密顿路径

$$s_{out}, v_{1,in}, v_{1,mid}, v_{1,out}, \dots, v_{n-2,in}, v_{n-2,mid}, v_{n-2,out}, t_{in}$$

而对于 UH 问题中的图 G' , 从 s_{out} 出发只能到达 $v_{a,in}$; 从 $v_{a,in}$ 出发只能前往 $v_{a,mid}$, 因为不存在能够到达 $v_{a,mid}$ 的其它合法路径; 从 $v_{a,mid}$ 出发只能前往 $v_{a,out}$, 因为这是唯一的与 $v_{a,mid}$ 相邻且未访问过的结点; 从 $v_{a,out}$ 出发只能前往 $v_{b,in} (b \neq a)$, 因为 $v_{a,out}$ 只与 $v_{a,mid}$ 和 $v_{b,in}$ 相邻, 而 $v_{a,mid}$ 已经访问过了; 以此类推直到路径到达 t_{in} , 当 UH 问题中的图 G' 中存在从 s_{out} 到 t_{in} 的哈密顿路径时, H 问题中的图 G 中存在从 s 到 t 的哈密顿路径; 所以 H 问题中的图 G 中存在从 s 到 t 的哈密顿路径, 当且仅当 UH 问题中的图 G' 中存在从 s_{out} 到 t_{in} 的哈密顿路径

由于可在关于 H 问题的结点数和边数的多项式时间内完成上述构造, 所以 $H \leq_P UH$, 进而 $UH \in \text{NP} - \text{C}$

定理 子集合之和问题 $SUBSET - SUM = \{\langle S, t \rangle | S \subseteq \mathbb{Z} \wedge \exists T \subseteq S, \sum_{x \in T} x = t\} \in \text{NP} - \text{C}$

 $SUBSET - SUM \in \text{NP}$ 证明

随机选择 $T \subseteq S$ 并验证 $\sum_{x \in T} x = t$, 可在多项式时间内验证 $SUBSET - SUM$ 问题

 $SUBSET - SUM \in \text{NP} - \text{C}$ 证明: $3 - SAT \leq_P SUBSET - SUM$

设 $3 - SAT$ 问题中的合取范式包含 k 个子句和 n 个不同变元, 则构造 $2n + 2k$ 行 $n + k$ 列的表格, 其中前 $2n$ 行对应 $3 - SAT$ 问题中的变元, 后 $2k$ 行对应 $3 - SAT$ 问题中的子句; 前 n 列对应 $3 - SAT$ 问题中的变元, 后 k 列对应 $3 - SAT$ 问题中的子句

对于 $3 - SAT$ 问题中的变元 x_i , 在第 $2i - 1$ 行第 i 列和第 $2i$ 行第 i 列分别填入 1;

若 x_i 在子句 f_j 中以正文字出现, 则在第 $2i - 1$ 行第 $n + j$ 列填入 1; 若 x_i 在子句 f_j 中以负文字出现, 则在第 $2i$ 行第 $n + j$ 列填入 1

对于 $3 - SAT$ 问题中的子句 f_j , 在第 $2n + 2j - 1$ 行第 $n + j$ 列和第 $2n + 2j$ 行第 $n + j$ 列分别填入 1

将表格中的每行视为一个 $n + k$ 位的十进制数, 构造 S 包含表格中的所有行对应的数; 构造 t 的前 n 位为 1, 后 k 位为 3, 则要使 $\sum_{x \in T} x = t$, 必须选择 T 中的数对应的行中, 前 n 列至多有 1 个 1, 即每个变元只能取真或假其中之一; 对于后 k 列, 每列必须有 3 个 1, 因为第 $2n + 2j - 1$ 行到第 $2n + 2j$ 行至多能提供 2 个 1, 则至少要包含 1 个来自子句 f_j 中变元 x_i 或 $\bar{x}_i$ 的 1, 即每个子句至少有 1 个文字为真

所以 $3 - SAT$ 问题中的合取范式可满足, 当且仅当 $SUBSET - SUM$ 问题中的 S 中存在子集 T 使得 $\sum_{x \in T} x = t$ (说明: 由于每列至多只有 5 个 1, 所以不存在进位问题)

由于可在关于 $3 - SAT$ 问题的子句数和变元数的多项式时间内完成上述构造, 所以 $3 - SAT \leq_p SUBSET - SUM$, 进而 $SUBSET - SUM \in NP - C$

3.2 空间复杂性

3.2.1 空间复杂度的表述

空间复杂度的表述：

- 设 TM M ，可能的输入为 Σ^* ，且假设 M 在任意输入上均能停机，对于任意输入 w 且 $|w| = n$ ， M 在 w 上使用的空间不超过 $f(n)$ ，则称 M 的空间复杂度为 $f(n)$
- 设 NTM N ，可能的输入为 Σ^* ，且假设 N 在任意输入上均能停机，对于任意输入 w 且 $|w| = n$ ， $N\langle w \rangle$ 的格局树中任意路径上使用的空间不超过 $f(n)$ ，则称 N 的空间复杂度为 $f(n)$

3.2.2 超线性空间复杂度类

PSPACE 类：在 $O(n^k)$, $k \in \mathbf{N}$ 空间内可在确定图灵机上判定的问题

NSPACE 类：在 $O(n^k)$, $k \in \mathbf{N}$ 空间内可在非确定图灵机上判定的问题

定理 $SAT \in \text{SPACE}(O(n))$

$SAT \in \text{PSPACE}$ 证明

首先存储合取范式 φ ，设 φ 包含 l 个不同变元，可用 l 个单位空间存储这些变元的赋值，之后用 φ 的空间判定 φ 是否可满足即可。由于变元数量级不超过输入长度，因此总空间复杂度为 $O(n)$ ，即 $SAT \in \text{SPACE}(O(n))$

定理 $\overline{ALL_{NFA}} = \{\langle A \rangle \mid A \text{ is NFA and } L(A) \neq \Sigma^*\} \in \text{NSPACE}(O(n))$

$\overline{ALL_{NFA}} \in \text{NSPACE}(O(n))$ 证明

随机猜测输入的字符至多 $2^{|Q|}$ 次，其中 Q 是输入 NFA A 的状态集合，将猜测的字符输入到 A 中，得到可能状态的集合，并判断该状态集是否包含 A 的接受状态，若不包含接受状态，则接受 A

由于每次得到的状态集都是 Q 的子集，因此状态集的数量级不超过 $2^{|Q|}$ ，而每个状态集的空间复杂度为 $O(|Q|)$ ，只需同时存储 2 个状态集以及前进步数的二进制计数，因此总空间复杂度为 $O(n)$ ，即 $\overline{ALL_{NFA}} \in \text{NSPACE}(O(n))$

萨维奇定理 (Savitch's Theorem) $\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f^2(n))$ ，其中 $f(n) \geq \log n$

萨维奇定理证明

设 $L \in \text{NSPACE}(f(n))$ ，则存在 NTM N ，使得 N 在任意输入上均能停机，并且对于任意输入 w 且 $|w| = n$ ， $N\langle w \rangle$ 的格局树中任意路径上使用的空间不超过 $f(n)$

因此 $N\langle w \rangle$ 的格局树的深度不超过 $2^{f(n)}$, 允许的时间复杂度也不超过 $2^{O(f(n))}$

只要利用 TM 验证 $N\langle w \rangle$ 是否在 $f(n)$ 空间与 $2^{O(f(n))}$ 时间内完成即可

对 N 重构为: N 在接受任意输入 w 前清除条带上的所有内容, 即格局为 $q_{ac}\sqcup$

在 TM 上构造算法 $CANYIELD(c_1, c_2, t)$, 其中 c_1, c_2 是 N 的格局, t 是正整数, $CANYIELD(c_1, c_2, t)$ 的功能是判断 N 是否能在 t 步内从格局 c_1 转变为格局 c_2 , 具体实现如下:

1. 若 $t = 1$, 则判断 c_1 是否能 1 步转变为 c_2 , 若能则输出接受, 否则输出拒绝
2. 若 $t > 1$, 则枚举 N 的占用空间不超过 $f(n)$ 的所有格局 c_m , 并递归调用 $CANYIELD(c_1, c_m, t/2)$ 和 $CANYIELD(c_m, c_2, t/2)$, 判断是否均输出接受, 若存在某个格局 c 使得两者均输出接受, 则输出接受, 否则输出拒绝

则只要调用 $CANYIELD(q_0\sqcup, q_{ac}\sqcup, 2^{O(f(n))})$ 即可判断 N 是否接受输入 w , 由于每次调用 $CANYIELD$ 的空间复杂度为 $O(f(n))$, 且递归深度为 $O(\log 2^{O(f(n))}) = O(f(n))$, 因此总空间复杂度为 $O(f^2(n))$, 即 $\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f^2(n))$

定理 $\text{PSPACE} = \text{NPSPACE}$

可由萨维奇定理得到 $\text{NPSPACE} = \text{NSPACE}(O(n^k)) \subseteq \text{SPACE}(O((n^k)^2)) \subseteq \text{PSPACE}$, 而显然有 $\text{PSPACE} \subseteq \text{NPSPACE}$, 因此 $\text{PSPACE} = \text{NPSPACE}$

总结: $\text{P} \subseteq \text{NP} \subseteq \text{PSPACE} = \text{NPSPACE} \subseteq \text{EXPTIME}$

3.2.3 PSPACE 完全性

PSPACE 难: 若问题 B 满足: $\forall A \in \text{PSPACE}, A \leq_p B$, 则称 B 为 PSPACE 难问题, 即 $B \in \text{PSPACE} - \text{H}$

PSPACE 完全: 若问题 B 满足: $B \in \text{PSPACE}$ 且 $B \in \text{PSPACE} - \text{H}$, 则称 B 为 PSPACE 完全问题, 即 $B \in \text{PSPACE} - \text{C}$

说明: 为什么归约过程限制为多项式时间而非多项式空间? 因为如果归约过程允许使用多项式空间, 那么归约过程本身使用的空间可能已经足够解决原问题了, 因此归约过程的空间复杂度必须限制在多项式时间内, 以保证归约过程不会过于强大, 从而使得 PSPACE 完全问题的定义具有实际意义

定理 $\text{TQBF} = \{\langle \varphi \rangle \mid \varphi \text{ is a true quantified boolean formula}\} \in \text{PSPACE} - \text{C}$

$\text{TQBF} \in \text{PSPACE}$ 证明

可用递归算法 $\text{Rec}\langle \varphi \rangle$ 判断输入的量化布尔公式 φ 是否为真, 具体实现如下:

1. 若 φ 无量词, 则直接判断 φ 是否为真, 若为真则输出接受, 否则输出拒绝

2. 若 φ 的最外层量词为 $\exists x$, 则递归调用 $Rec\langle\varphi[x/\text{true}]\rangle$ 和 $Rec\langle\varphi[x/\text{false}]\rangle$, 判断是否至少有 1 个输出接受, 若是则输出接受, 否则输出拒绝
3. 若 φ 的最外层量词为 $\forall x$, 则递归调用 $Rec\langle\varphi[x/\text{true}]\rangle$ 和 $Rec\langle\varphi[x/\text{false}]\rangle$, 判断是否均输出接受, 若是则输出接受, 否则输出拒绝

由于每次递归调用 Rec 的空间复杂度为 $O(1)$, 且递归深度不超过输入 φ 中的量词数量, 因此总空间复杂度为 $O(n)$, 即 $TQBF \in PSPACE$

$TQBF \in PSPACE - H$ 证明: $\forall A \in PSPACE, A \leq_p TQBF$

可借助萨维奇定理的证明过程, 设范式 $\varphi_{c_1, c_2, t}$ 为真当且仅当 $CANYIELD(c_1, c_2, t)$ 输出接受, 则可在多项式时间内将任意 $A \in PSPACE$ 的计算历史归约为 $\varphi_{q_0 \sqcup, q_{ac} \sqcup, 2^{n^k}}$:

$$\varphi_{c_1, c_2, t} = \exists c_m : \varphi_{c_1, c_m, t/2} \wedge \varphi_{c_m, c_2, t/2}$$

其中 $\exists c_m$ 表示枚举 N 的占用空间不超过 n^k 的所有格局 c_m , 可以重写为:

$$\bigwedge_{i=1}^{n^k} \left(\bigvee_{j \in Q \cup \Gamma \cup \{\sqcup\}} \text{The content of position } i \text{ is } j \right)$$

此时 $\varphi_{q_0 \sqcup, q_{ac} \sqcup, 2^{n^k}}$ 完全展开依然为指数空间, 可将 $\varphi_{c_1, c_2, t}$ 转化为

$$\exists m : \forall (c_3, c_4) \in \{(c_1, m), (m, c_2)\} : \varphi_{c_3, c_4, t/2}$$

这样, 递归层数每增加 1 时, 只需添加 $\forall (c_3, c_4) \in \{(c_1, m), (m, c_2)\}$ 即可, 且每层递归只需存储 1 个格局 m , 因此总空间复杂度为 $n^{O(k)}$, 且可在多项式时间内完成归约

因此 $A \leq_p TQBF$, 即 $TQBF \in PSPACE - H$, 综上 $TQBF \in PSPACE - C$

根据 $TQBF \in PSPACE - C$ 可知: 设 φ 是量化的合取范式, 满足: 首个量词为 $\exists$, 且量词交替出现, 且最后的量词为 $\exists$, 有 2 名玩家 E 和 A 轮流为 φ 中的变元赋值, 玩家 E 的目标是使 φ 为真, 而玩家 A 的目标是使 φ 为假, 有 $\varphi \in TQBF$ 当且仅当玩家 E 有必胜策略 (过程中可为 $TQBF$ 问题中的合取范式插入不影响结果的哑元变量), 因此 E 是否有必胜策略是 $PSPACE$ 完全问题

例 设有向无环图 G , 从结点 s 出发, 2 名玩家 A 和 B 轮流选择 G 中的边前进, 且过程中不允许选择已经选择的结点, 当某个玩家无法继续前进时, 该玩家输掉游戏

证明: 判断玩家 A 是否有必胜策略是 $PSPACE$ 完全问题

解 归约: $TQBF$ 游戏 $\leq_p G$ 游戏

设 $TQBF$ 游戏中的量化布尔公式为 φ , 设其包含 n 个不同变元 $x_1, x_2, \dots, x_n$ 和 k 个析取子式 $c_1, c_2, \dots, c_m$, 则可在多项式时间内构造 G , 使得玩家 A 在 G 游戏中有必

胜策略当且仅当玩家 E 在 $TQBF$ 游戏中有必胜策略，具体构造如下：

- 对于 φ 中的每个变元 x_i ，构造 G 中的 4 个结点 $x_i^{in}, x_i^T, x_i^F, x_i^{out}$ ，并用边连接 $(x_i^{in}, x_i^T), (x_i^{in}, x_i^F), (x_i^T, x_i^{out}), (x_i^F, x_i^{out})$ 以及 $(x_{i-1}^{out}, x_i^{in}), i = 2, 3, 4, \dots, n$ ，其中 x_1^{in} 即为 G 中的结点 s
- 对于 φ 中的每个析取子式 c_j ，构造 G 中的 1 个结点 c_j ；设 $c_j = \bigvee_{i \in [n]} w_i$ ，其中 w_i 是 x_i 或 $\neg x_i$ ，构造结点 w_i ，并用边连接 (c_j, w_i) ；对于 $w_i = x_i$ ，用边连接 (w_i, x_i^T) ；对于 $w_i = \neg x_i$ ，用边连接 (w_i, x_i^F)
- 构造 G 中的结点 c ，并用边连接 $(c, c_j), j = 1, 2, \dots, m$ 以及 (x_n^{out}, c)

游戏的过程为：

1. 玩家 A 从结点 $s = x_1^{in}$ 出发，可以选择边 (x_1^{in}, x_1^T) 或 (x_1^{in}, x_1^F) 前进到结点 x_1^T 或 x_1^F ，玩家 B 只能根据玩家 A 的选择前进到结点 x_1^{out} ，玩家 A 只能选择前进到 x_2^{in}
2. 玩家 B 从结点 x_2^{in} 出发，可以选择边 (x_2^{in}, x_2^T) 或 (x_2^{in}, x_2^F) 前进到结点 x_2^T 或 x_2^F ，玩家 A 只能根据玩家 B 的选择前进到结点 x_2^{out} ，玩家 B 只能选择前进到 x_3^{in}
3. 依此类推，玩家 A 和 B 轮流选择前进到 x_i^T 或 x_i^F ，之后前进到 x_i^{out} ，再前进到 x_{i+1}^{in} ，直到玩家 A 从结点 x_n^{in} 前进到 x_n^T 或 x_n^F ，之后玩家 B 只能前进到 x_n^{out} ，玩家 A 只能前进到结点 c
4. 此时玩家 B 可以任意选择前进到某个结点 c_j
 - 若 $TQBF$ 游戏中的 φ 可满足，则 $\forall c_j, \exists w_i : w_i = T$ ，则玩家 A 可以前进到结点 w_i ，之后由于使得 $w_i = T$ 的结点 x_i^T 或 x_i^F 已经被选择过，因此玩家 B 无法继续前进，玩家 A 获胜
 - 若 $TQBF$ 游戏中的 φ 不可满足，则 $\exists c_j, \forall w_i : w_i = F$ ，则玩家 B 可以前进到结点 c_j ，之后由于 $\forall i$ ，使得 $w_i = F$ 的结点 x_i^T 或 x_i^F 均未被选择，则玩家 B 可以前进到 x_i^T 或 x_i^F ，而 x_i^{out} 已经被选择过，因此玩家 A 无法继续前进，玩家 B 获胜

因此玩家 A 在 G 游戏中有必胜策略当且仅当玩家 E 在 $TQBF$ 游戏中有必胜策略，即 $TQBF$ 游戏 $\leq_p G$ 游戏，因此判断玩家 A 是否有必胜策略是 PSPACE 完全问题

3.2.4 亚线性空间复杂度类

对图灵机模型做出的修改：亚线性空间无法存储输入，因此对图灵机模型做出修改：

- 输入带：只读，不可写入，长度为输入长度，且在输入的任意位置上均可进行读操作
- 工作带：可读写，但空间复杂度受限，为 $o(n)$

L 类：工作带的空间复杂度为 $O(\log n)$ 时可在确定图灵机上判定的问题

NL 类：工作带的空间复杂度为 $O(\log n)$ 时可在非确定图灵机上判定的问题

例 证明: $A = \{0^n 1^n \mid n \geq 0\} \in L$

解 首先判定是否符合 $0^n 1^n$ 的形式, 然后可在对数空间内使用二进制计数器记录 0 的数量, 之后判断 1 的数量是否与 0 的数量相同, 若相同则接受, 否则拒绝, 因此 $A \in L$

定理 $PATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph and there is a path from } s \text{ to } t\} \in NL$

$PATH \in NL$ 证明

随机猜测 G 中的一个结点 v , 并判断 v 是否为 t , 若是则接受, 否则继续猜测 G 中的一个结点 v' , 并判断 (v, v') 是否为 G 中的边, 若是则将 v' 赋值给 v , 继续上述过程, 直到猜测的结点数量达到 $|V|$, 其中 V 是 G 的结点集合

猜测的结点数量不能超过 $|V|$, 是因为如果猜测的结点数量超过 $|V|$, 则必然存在重复的结点, 即存在环路, 与连通性无关, 因此不需要继续猜测下去, 直接拒绝即可

由于每次猜测的结点 v 和 v' 以及计数器的空间复杂度为 $O(\log n)$, 且总共猜测的结点数量不超过 $|V|$, 因此总空间复杂度为 $O(\log n)$, 即 $PATH \in NL$

注意: 此处使用的图灵机模型对 $TIME(2^{O(f(n))}) \subseteq SPACE(f(n))$ 不再成立, 例如: 仅把输入带遍历但不使用工作带, 则时间复杂度为 $O(n)$, 但空间复杂度为 $O(1)$, 显然 $n \neq 2^{O(1)}$
对格局的扩充定义: 引入输入带和工作带的区分后, 此时图灵机的格局包括图灵机的状态、工作带的内容、工作带的指针位置、输入带的指针位置

有: 若工作带的空间复杂度为 $f(n)$, 则图灵机不同的格局数量为 $|Q| \cdot |\Gamma|^{f(n)} \cdot f(n) \cdot n = n \cdot 2^{O(f(n))}$, 其中 Q 是图灵机的状态集合, Γ 是工作带的符号集合

对于 $f(n) = \Omega(\log n)$ 的情况, 可证明仍然有 $TIME(2^{O(f(n))}) \subseteq SPACE(f(n))$

3.2.5 NL 完全性

对数空间归约: 设 A 和 B 是语言, 若存在函数 f , 使得对于任意输入 w , $w \in A$ 当且仅当 $f(w) \in B$, 且 f 可在对数空间内计算, 则称 A 对数空间归约到 B , 记为 $A \leq_L B$

其中对数空间可计算指: 输入带和输出带无限, 但工作带的长度被限制为 $O(\log n)$, 且在计算过程中输出带的内容不能被修改; 即输入带只读, 工作带可读写, 但空间复杂度受限为 $O(\log n)$, 输出带只能写, 但不能读和修改

NL 难: 若问题 B 满足: $\forall A \in NL, A \leq_L B$, 则称 B 为 NL 难问题, 即 $B \in NL - H$

NL 完全: 若问题 B 满足: $B \in NL$ 且 $B \in NL - H$, 则称 B 为 NL 完全问题, 即 $B \in NL - C$

定理 若 $A \leq_L B$ 且 $B \in L$, 则 $A \in L$

对数空间归约的传递性证明

采用时间换空间的思想，设 $x \in \Sigma_A^*$ 是 A 的输入， f 是 A 到 B 的对数空间归约函数，且 f 可在对数空间内计算，则直接调用 B 的对数空间算法，当需要 $f(x)$ 的第 i 个字符时，调用 f 的对数空间算法来计算 $f(x)$ 的第 i 个字符

由于计算 f 的空间复杂度为 $O(\log n)$ ，且判定 B 的空间复杂度为 $O(\log n)$ ，因此总空间复杂度为 $O(\log n)$ ，即 $A \in L$

定理 $PATH \in NL - C$

我们已经证明了 $PATH \in NL$ ，因此只需证明 $PATH \in NL - H$ 即可

$\forall A \in NL, A \leq_L PATH$ 证明

设 $A \in NL$ ，则存在 NTM N ，使得 N 可在 $O(\log n)$ 空间内判定 A

设 $\langle G, s, t \rangle \in \Sigma_{PATH}^*$ ，其中 $G = (V, E)$ ，将 V 构造为 N 的所有格局的集合， E 构造为 N 的合法状态转移，即 $(c_1, c_2) \in E$ 当且仅当 N 可以从格局 c_1 1 步转变为格局 c_2

归约过程中，只需要遍历 N 的所有格局以构造 V ，再双重遍历 N 的所有格局，每次检测对应的 2 个格局之间是否存在边即可，因此归约过程的空间复杂度为 $O(\log n)$

最后，将 $\langle G, s, t \rangle$ 作为 $PATH$ 的实例，其中 $s = c_0, t = c_{ac}$ 即可完成归约，即 $A \leq_L PATH$ ，因此 $PATH \in NL - C$

定理 $NL \subseteq P$

NL $\subseteq$ P 证明

$\forall A \in NL, A \leq_L PATH$ ，且 $PATH \in P$

由于归约过程是在对数空间内完成的，则必然可在多项式时间内完成，因此 $A \in P$

定理 $NL = co - NL$

首先考虑 $\overline{PATH} - c$ 问题： $\overline{PATH} - c = \{\langle G, s, t, c \rangle \mid G \text{ is a directed graph} \wedge \text{there is no path from } s \text{ to } t \wedge \text{and there are exactly } c \text{ vertices that are reachable from } s\}$

$\overline{PATH} - c \in NL$ 证明

首先，遍历所有结点，对于每个结点 v ，随机指定从 s 出发是否可以到达 v

- 若指定从 s 出发不能到达 v ，则继续遍历下个结点
- 若指定从 s 出发可以到达 v ，则利用 $PATH$ 的 NL 算法验证 v 是否可达
 - 若 v 不可达，则直接拒绝

– 若 v 可达，则将计数器加 1，并继续遍历下个结点

遍历全部结点后，若计数器的值等于 c 且从 s 出发不可达 t ，则接受，否则拒绝

由于每次验证 $PATH$ 的空间复杂度为 $O(\log n)$ ，且计数器的值不超过 $O(n)$ ，对应计数器需要 $O(\log n)$ 的空间，因此总空间复杂度为 $O(\log n)$ ，即 $\overline{PATH} - c \in NL$

$\overline{PATH} - c$ 中 c 的求解

设点集 $A_i = \{v | v \text{ can be reached from } s \text{ in at most } i \text{ steps}\}$ ，设 $c_i = |A_i|$ ，则只要求出 $c = c_{|V|-1}$ 即可，因为 G 中的最长的无环路径长度不超过 $|V| - 1$

首先 $A_0 = \{s\}$ ，所以 $c_0 = 1$ 显然；之后，若 v_i 满足 $\exists v_j \in A_k$ 且 $(v_j, v_i) \in E$ ，则 $v_i \in A_{k+1}$ ；假设 c_k 已经求出，则在求 c_{k+1} 的过程中，对某个结点 v_i ，遍历所有结点 v_j ，首先随机指定 v_j 是否属于 A_k

- 若指定 v_j 不属于 A_k ，则继续遍历下个结点
- 若指定 v_j 属于 A_k ，则利用 $PATH$ 的 NL 算法验证从 s 出发 k 步内是否可达 v_j ，以及 (v_j, v_i) 是否为 G 中的边
 - 若 k 步内不可达或 (v_j, v_i) 不为 G 中的边，则直接拒绝
 - 若 k 步内可达且 $(v_j, v_i) \in E$ ，则将记录 c_{k+1} 的值加 1，并遍历下个结点

因此可以在 NL 中求出 $c = c_{|V|-1}$ ，之后调用 $\overline{PATH} - c$ 的 NL 算法判断 $\langle G, s, t, c \rangle$ 是否接受，即可判断 $\langle G, s, t \rangle$ 是否属于 $\overline{PATH}$ ，因此 $\overline{PATH} \in NL$

$\overline{PATH} \in NL \Rightarrow NL = co - NL$ 证明

首先证明 $co - NL \subseteq NL$:

$\because PATH \in NL - C$

$\therefore \forall A \in NL, A \leq_L PATH$

$\therefore \forall B \in co - NL, \overline{B} \in NL, \overline{B} \leq_L PATH, B \leq_L \overline{PATH}$

$\therefore B \in NL$

$\therefore co - NL \subseteq NL$

然后证明 $NL \subseteq co - NL$:

$\because PATH \in NL$

$\therefore \overline{PATH} \in co - NL$

$\because PATH \in NL - C$

$\therefore \forall A \in NL, A \leq_L PATH$

$\therefore A \in co - NL$

$\therefore NL \subseteq co - NL$

综上 $NL = co - NL$

复杂性总结: $L \subseteq NL = co - NL \subseteq P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME$

4 (*)计算理论中的其他论题

4.1 分层定理

4.1.1 空间分层定理

空间可构造: 若 $f(n) = \Omega(\log n)$, 且 $f(n)$ 能够在 $O(f(n))$ 的空间内用二进制表示出来, 则称 $f(n)$ 是空间可构造的

空间分层定理 若 $f(n)$ 是空间可构造的, 则存在语言 A , 使得 $A \in \text{SPACE}(f(n))$, 但 $A \notin \text{SPACE}(o(f(n)))$

空间分层定理证明 构造 TM D , 令 $D\langle M \rangle$:

1. 计算 $f(|M|)$, 并用 $O(f(|M|))$ 空间模拟 $M\langle M \rangle$
2. 如果 $M\langle M \rangle$ 在 $f(|M|)$ 空间内不停机, 则 D 输出拒绝
3. 如果 $M\langle M \rangle$ 在 $f(|M|)$ 空间内停机, 则 D 将 $M\langle M \rangle$ 的结果取反后输出

显然 D 在 $O(f(n))$ 空间内停机, 因此 $D \in \text{SPACE}(O(f(n)))$

根据对角线方法, 设 M_i 代表能够在 $o(f(n))$ 空间内停机的图灵机, 则 D 与 M_i 在输入 $\langle M_i \rangle$ 上的结果不同, 因此 D 与任意 M_i 均不同, 因此 $D \notin \text{SPACE}(o(f(n)))$, 即证

推论 若 $f_1(n) = o(f_2(n))$, 则 $\text{SPACE}(f_1(n)) \subset \text{SPACE}(f_2(n))$

例 证明: $\text{NL} \subset \text{PSPACE}$

证明 根据萨维奇定理, $\text{NL} \subseteq \text{SPACE}(\log^2 n)$

再根据空间分层定理, $\text{SPACE}(\log^2 n) \subset \text{SPACE}(n^k)$

因此 $\text{NL} \subset \text{PSPACE}$, 即证

推论 $\text{TQBF} \notin \text{NL}$

4.1.2 时间分层定理

时间可构造: 若 $f(n) = \Omega(n \log n)$, 且能够在 $O(f(n))$ 的时间内计算出 $f(n)$ 的二进制表示, 则称 $f(n)$ 是时间可构造的

时间分层定理 若 $f(n)$ 是时间可构造的, 则存在语言 A , 使得 $A \in \text{TIME}(f(n))$, 但 $A \notin \text{TIME}\left(o\left(\frac{f(n)}{\log f(n)}\right)\right)$

推论 若 $f_1(n) = o\left(\frac{f_2(n)}{\log f_2(n)}\right)$, 则 $\text{TIME}(f_1(n)) \subset \text{TIME}(f_2(n))$

推论 若 $1 \leq \varepsilon_1 \leq \varepsilon_2$, 则 $\text{TIME}(n^{\varepsilon_1}) \subset \text{TIME}(n^{\varepsilon_2})$

4.2 对角线方法的局限性

定理 对角线方法无法证明 $P = NP$ 或 $P \neq NP$

对角线方法局限性证明

设 M^A 代表某种神化图灵机， M 是图灵机模型，且可以询问当前指针指向格子的符号是否属于集合 A ，并且可以在 $O(1)$ 时间内得到输入字符串是否属于 A 的答复

设 $P^A = \{\langle M^A \rangle | M \in P\}$, $NP^A = \{\langle M^A \rangle | M \in NP\}$

则有: $\forall A, P^A = NP^A \Rightarrow P = NP$ 且 $\forall A, P^A \neq NP^A \Rightarrow P \neq NP$

然而，可构造集合 A ，使得 $P^A = NP^A$ ，又存在集合 B ，使得 $P^B \neq NP^B$ ，所以上述 2 个蕴含关系均无法证明 $P = NP$ 或 $P \neq NP$ ，即证

4.3 逻辑电路复杂性

由于给定的逻辑电路拥有固定的输入个数，因此逻辑电路复杂性考虑的是电路族，包括深度和规模两个维度

定理 若 $A \in \text{TIME}(f(n))$ ，则 A 的逻辑电路复杂度为 $O(f^2(n))$

定理 逻辑电路可满足问题是 NP 完全问题